

drag-lint

Symbol-aware index, RAG and lint for Delphi / Object Pascal

Version 1.5.0-alpha

Generated 2026-08-17

Part I -- Getting started	22
drag-lint.....	23
Start here	23
The one-paragraph model	23
Two commands that answer most questions.....	23
A trap worth knowing on day one	24
Links	24
Installation	25
1. CLI.....	25
2. Build your first index.....	26
3. RAD Studio plugin	26
4. Editors (LSP)	27
Suppressing a finding	27
Maintenance	28
Where indexes live.....	28
The manifest	28
Routine reindexing.....	29
Reading a long run	29
Troubleshooting.....	30
Third-party library updates.....	31
Features	32
Indexing.....	32
Search and navigation (<i>index</i>).....	33
Linting.....	33
Documentation	35
Refactoring and code generation	35

Component conversion	35
Graphs and reports	36
Compiler integration	36
Database and Firebird	37
Editor integration	37
Maintenance and diagnostics	37
Part II -- The RAD Studio plugin	38
IDE Menu Reference	39
Pinned at the top	39
Panels	39
Everyday actions	39
Uses && Dependencies	40
Inspect Symbol	41
Code Quality	41
Generate && Export	41
Index && Maintenance	42
Compile && Analysis	42
About	42
View > Tool Windows	43
About and Status	44
Why it exists	44
The groups	44
Actions	45
Performance	45
The connection indicator	46
See also	46
Add Missing Units to uses (whole unit)	47
Running it from the CLI	47
Reaching it in the IDE	47
What it needs	47
Example	47
Allow this message	48
Reaching it in the IDE	48

Running it from the CLI	48
What it needs.....	48
Example.....	48
Auto-Document Whole Project	49
Running it from the CLI	49
Reaching it in the IDE	49
What it needs.....	49
Example.....	49
Call Graph (Butterfly)	50
Running it from the CLI	50
Reaching it in the IDE	50
What it needs.....	50
Example.....	50
Circular Uses Report (cycles + fix plan)	52
Running it from the CLI	52
Reaching it in the IDE	52
What it needs.....	52
Example.....	52
Class Surface... ..	53
Reaching it in the IDE	53
Running it from the CLI	53
What it needs.....	53
Example.....	53
Compile Buffer (unsaved)	54
Running it from the CLI	54
Reaching it in the IDE	54
What it needs.....	54
Example.....	54
Compile & Diagnose.....	55
Running it from the CLI	55
Reaching it in the IDE	55
What it needs.....	55
Example.....	55

Compiler Hints	56
Running it from the CLI	56
Reaching it in the IDE	56
What it needs.....	56
Example.....	56
Copy All Diagnostics	57
Reaching it in the IDE	57
Running it from the CLI	57
What it needs.....	57
Example.....	57
Copy Diagnostics (Current File).....	58
Running it from the CLI	58
Reaching it in the IDE	58
What it needs.....	58
Example.....	58
Create helper class.....	59
Reaching it in the IDE	59
Running it from the CLI	59
What it needs.....	59
Example.....	59
Doc Comment Stub (symbol)	60
Running it from the CLI	60
Reaching it in the IDE	60
What it needs.....	60
Example.....	60
Document it	61
Reaching it in the IDE	61
Running it from the CLI	61
What it needs.....	61
Example.....	61
Document project	62
Reaching it in the IDE	62
Running it from the CLI	62

What it needs.....	62
Example.....	62
Document unit	63
Reaching it in the IDE	63
Running it from the CLI	63
What it needs.....	63
Example.....	63
drag-lint.....	64
Reaching it in the IDE	64
Running it from the CLI	64
What it needs.....	64
Example.....	64
drag-lint Graph.....	65
Reaching it in the IDE	65
Running it from the CLI	65
What it needs.....	65
Example.....	65
drag-lint Graph (dockable).....	66
Reaching it in the IDE	66
Running it from the CLI	66
What it needs.....	66
Example.....	66
drag-lint Options	67
Running it from the CLI	67
Reaching it in the IDE	67
What it needs.....	67
Example.....	67
drag-lint Panel (dockable).....	68
Reaching it in the IDE	68
Running it from the CLI	68
What it needs.....	68
Example.....	68
drag-lint: Project Rules.....	69

Reaching it in the IDE	69
Running it from the CLI	69
What it needs.....	69
Example.....	69
Export Enums (Delphi const).....	70
Running it from the CLI	70
Reaching it in the IDE	70
What it needs.....	70
Example.....	70
Export Graph (DOT).....	71
Running it from the CLI	71
Reaching it in the IDE	71
What it needs.....	71
Example.....	71
Export to Obsidian	72
Running it from the CLI	72
Reaching it in the IDE	72
What it needs.....	72
Example.....	72
Find Dead Code	73
Running it from the CLI	73
Reaching it in the IDE	73
What it needs.....	73
Example.....	73
Find Undocumented (public)	74
Running it from the CLI	74
Reaching it in the IDE	74
What it needs.....	74
Example.....	74
Find Usages... ..	75
Reaching it in the IDE	75
Running it from the CLI	75
What it needs.....	75

Example.....	75
Find Usages (context).....	76
Reaching it in the IDE	76
Running it from the CLI	76
What it needs.....	76
Example.....	76
Fix all in project.....	77
Reaching it in the IDE	77
Running it from the CLI	77
What it needs.....	77
Example.....	77
Fix all in unit	79
Reaching it in the IDE	79
Running it from the CLI	79
What it needs.....	79
Example.....	79
Fix it.....	81
Reaching it in the IDE	81
Running it from the CLI	81
What it needs.....	81
Example.....	81
Format Whole Project with YADF	83
Running it from the CLI	83
Reaching it in the IDE	83
What it needs.....	83
Formatting does NOT invalidate your review markers.....	83
Example.....	84
Format with YADF	85
Running it from the CLI	85
Reaching it in the IDE	85
What it needs.....	85
Example.....	85
Full Compile Sweep.....	86

Running it from the CLI	86
Reaching it in the IDE	86
What it needs.....	86
Example.....	86
Generate Test Helper CSV.....	87
Running it from the CLI	87
Reaching it in the IDE	87
What it needs.....	87
Example.....	87
Go to Declaration	88
Reaching it in the IDE	88
Running it from the CLI	88
What it needs.....	88
Example.....	88
Go to Definition.....	89
Reaching it in the IDE	89
Running it from the CLI	89
What it needs.....	89
Example.....	89
Go to Implementation	90
Reaching it in the IDE	90
Running it from the CLI	90
What it needs.....	90
Example.....	90
Hover at Cursor	91
Reaching it in the IDE	91
Running it from the CLI	91
What it needs.....	91
Example.....	91
Impact / Blast Radius (symbol)	92
Running it from the CLI	92
Reaching it in the IDE	92
What it needs.....	92

Example.....	92
Import Build Log.....	93
Running it from the CLI	93
Reaching it in the IDE	93
What it needs.....	93
Example.....	93
Library Drift Check	94
Running it from the CLI	94
Reaching it in the IDE	94
What it needs.....	94
Example.....	94
Lint Buffer (Unsaved)	95
Running it from the CLI	95
Reaching it in the IDE	95
What it needs.....	95
Example.....	95
Open Plugin Log	96
Running it from the CLI	96
Reaching it in the IDE	96
What it needs.....	96
Example.....	96
Quick-Fix: Add Unit for Inline Hint (H2443) at Cursor	97
Running it from the CLI	97
Reaching it in the IDE	97
What it needs.....	97
Example.....	97
Quick-Fix: Add Unit for Undeclared at Cursor (Ctrl+Alt+U)	98
Running it from the CLI	98
Reaching it in the IDE	98
What it needs.....	98
Example.....	98
Quick-Fix: Convert Public Field to Property at Cursor	99
Running it from the CLI	99

Reaching it in the IDE	99
What it needs.....	99
Example.....	99
Rebuild Index for This Project.....	100
Running it from the CLI	100
Reaching it in the IDE	100
What it needs.....	100
Example.....	100
Reconcile Project Members (.dpr/.dproj).....	101
Running it from the CLI	101
Reaching it in the IDE	101
What it needs.....	101
Example.....	101
Recover Buffer-Compile Files.....	102
Running it from the CLI	102
Reaching it in the IDE	102
What it needs.....	102
Example.....	102
Rename Symbol... ..	103
Reaching it in the IDE	103
Running it from the CLI	103
What it needs.....	103
Example.....	103
Reverse Call Tree (clickable, Messages window).....	104
Running it from the CLI	104
Reaching it in the IDE	104
What it needs.....	104
Example.....	104
Reverse Call Tree (who calls this, N-deep).....	105
Running it from the CLI	105
Reaching it in the IDE	105
What it needs.....	105
Example.....	105

Run AST Checks	106
Running it from the CLI	106
Reaching it in the IDE	106
What it needs.....	106
Example.....	106
Run Diagnostics (didSave)	107
Running it from the CLI	107
Reaching it in the IDE	107
What it needs.....	107
Example.....	107
Run Lint All (Full Report)	108
Running it from the CLI	108
Reaching it in the IDE	108
What it needs.....	108
Example.....	108
Scan TODOs / FIXMEs.....	109
Running it from the CLI	109
Reaching it in the IDE	109
What it needs.....	109
Example.....	109
Show Completion.....	110
Reaching it in the IDE	110
Running it from the CLI	110
What it needs.....	110
Example.....	110
Show in Call Graph	111
Reaching it in the IDE	111
Running it from the CLI	111
What it needs.....	111
Example.....	111
Show Resolved DBs (debug)	112
Running it from the CLI	112
Reaching it in the IDE	112

What it needs.....	112
Example.....	112
Show Signature Help.....	113
Reaching it in the IDE	113
Running it from the CLI	113
What it needs.....	113
Example.....	113
Show Structure.....	114
Reaching it in the IDE	114
Running it from the CLI	114
What it needs.....	114
Example.....	114
Show Wiring (Spring4D DI + DFM events)	115
Running it from the CLI	115
Reaching it in the IDE	115
What it needs.....	115
Example.....	115
Symbol Search.....	116
Reaching it in the IDE	116
Running it from the CLI	116
What it needs.....	116
Example.....	116
Symbol Slice... ..	117
Reaching it in the IDE	117
Running it from the CLI	117
What it needs.....	117
Example.....	117
Top Symbols (fan-in)	118
Running it from the CLI	118
Reaching it in the IDE	118
What it needs.....	118
Example.....	118
Type at Cursor	119

Reaching it in the IDE	119
Running it from the CLI	119
What it needs.....	119
Example.....	119
Unit Test Stub (symbol).....	120
Running it from the CLI	120
Reaching it in the IDE	120
What it needs.....	120
Example.....	120
Uses Audit -- interface->impl moves + unused (this unit)	121
Running it from the CLI	121
Reaching it in the IDE	121
What it needs.....	121
Example.....	121
Uses Cleanup Preview (compiler-verified, this unit)	122
Running it from the CLI	122
Reaching it in the IDE	122
What it needs.....	122
Example.....	122
Uses Report (CSV)	123
Running it from the CLI	123
Reaching it in the IDE	123
What it needs.....	123
Example.....	123
Part III -- The command line.....	124
allow.....	125
Running it from the CLI	125
Reaching it in the IDE	125
What it needs.....	125
Example.....	125
ambiguous-calls	126
Running it from the CLI	126
Reaching it in the IDE	126

What it needs.....	126
Example.....	126
bench-context.....	127
Running it from the CLI	127
Reaching it in the IDE	127
What it needs.....	127
Example.....	127
butterfly	128
Running it from the CLI	128
Reaching it in the IDE	128
What it needs.....	128
Example.....	128
call-path	129
Running it from the CLI	129
Reaching it in the IDE	129
What it needs.....	129
Example.....	129
callgraph.....	130
Running it from the CLI	130
Reaching it in the IDE	130
What it needs.....	130
Example.....	130
compile-check.....	131
Running it from the CLI	131
Reaching it in the IDE	131
What it needs.....	131
Example.....	131
context	132
Running it from the CLI	132
Reaching it in the IDE	132
What it needs.....	132
Example.....	132
convert-apply	133

Running it from the CLI	133
Reaching it in the IDE	133
What it needs.....	133
Example.....	133
convert-scaffold	134
Running it from the CLI	134
Reaching it in the IDE	134
What it needs.....	134
Example.....	134
convert-validate	135
Running it from the CLI	135
Reaching it in the IDE	135
What it needs.....	135
Example.....	135
create-enum-helper	136
Running it from the CLI	136
Reaching it in the IDE	136
What it needs.....	136
Example.....	136
deps-report	137
Running it from the CLI	137
Reaching it in the IDE	137
What it needs.....	137
Example.....	137
diff	138
Running it from the CLI	138
Reaching it in the IDE	138
What it needs.....	138
Example.....	138
doc-drift	139
Running it from the CLI	139
Reaching it in the IDE	139
What it needs.....	139

Example.....	139
document-all	140
Running it from the CLI	140
Reaching it in the IDE	140
What it needs.....	140
Example.....	140
dump-call-edges.....	141
Running it from the CLI	141
Reaching it in the IDE	141
What it needs.....	141
Example.....	141
dump-refs.....	142
Running it from the CLI	142
Reaching it in the IDE	142
What it needs.....	142
Example.....	142
extract-method	143
Running it from the CLI	143
Reaching it in the IDE	143
What it needs.....	143
Example.....	143
fb-snapshot	144
Running it from the CLI	144
Reaching it in the IDE	144
What it needs.....	144
Example.....	144
find-callees	145
Running it from the CLI	145
Reaching it in the IDE	145
What it needs.....	145
Example.....	145
find-unit	146
Running it from the CLI	146

Reaching it in the IDE	146
What it needs.....	146
Example.....	146
ghost-check.....	147
Running it from the CLI	147
Reaching it in the IDE	147
What it needs.....	147
Example.....	147
ghost-recover	148
Running it from the CLI	148
Reaching it in the IDE	148
What it needs.....	148
Example.....	148
helpers-of.....	149
Running it from the CLI	149
Reaching it in the IDE	149
What it needs.....	149
Example.....	149
info	150
Running it from the CLI	150
Reaching it in the IDE	150
What it needs.....	150
Example.....	150
link-orm.....	151
Running it from the CLI	151
Reaching it in the IDE	151
What it needs.....	151
Example.....	151
lint-project	152
Running it from the CLI	152
Reaching it in the IDE	152
What it needs.....	152
Example.....	152

migrate-dbs	153
Running it from the CLI	153
Reaching it in the IDE	153
What it needs.....	153
Example.....	153
outline	154
Running it from the CLI	154
Reaching it in the IDE	154
What it needs.....	154
Example.....	154
pp-profile	155
Running it from the CLI	155
Reaching it in the IDE	155
What it needs.....	155
Example.....	155
preprocess-file	156
Running it from the CLI	156
Reaching it in the IDE	156
What it needs.....	156
Example.....	156
proptree	157
Running it from the CLI	157
Reaching it in the IDE	157
What it needs.....	157
Example.....	157
purge-locals.....	158
Running it from the CLI	158
Reaching it in the IDE	158
What it needs.....	158
Example.....	158
query ancestors.....	159
Running it from the CLI	159
Reaching it in the IDE	159

What it needs.....	159
Example.....	159
query find-callers	160
Running it from the CLI	160
Reaching it in the IDE	160
What it needs.....	160
Example.....	160
query typecat	161
Running it from the CLI	161
Reaching it in the IDE	161
What it needs.....	161
Example.....	161
safe-delete	162
Running it from the CLI	162
Reaching it in the IDE	162
What it needs.....	162
Example.....	162
schema	163
Running it from the CLI	163
Reaching it in the IDE	163
What it needs.....	163
Example.....	163
serve.....	164
Running it from the CLI	164
Reaching it in the IDE	164
What it needs.....	164
Example.....	164
shared-unit.....	165
Running it from the CLI	165
Reaching it in the IDE	165
What it needs.....	165
Example.....	165
usages	166

Running it from the CLI	166
Reaching it in the IDE	166
What it needs.....	166
Example.....	166
workspace add	167
Running it from the CLI	167
Reaching it in the IDE	167
What it needs.....	167
Example.....	167
workspace index	168
Running it from the CLI	168
Reaching it in the IDE	168
What it needs.....	168
Example.....	168
workspace status	169
Running it from the CLI	169
Reaching it in the IDE	169
What it needs.....	169
Example.....	169
Part IV -- Reference.....	170
rules.....	171
Running it from the CLI	171
Reaching it in the IDE	171
What it needs.....	171
Example.....	171
lsp.....	172
Running it from the CLI	172
Reaching it in the IDE	172
What it needs.....	172
Example.....	172
Feature Index	173
Main menu	173
Right-click menus	174

Tool windows	174
CLI verbs	174
Circular Dependency Report	176
The sample project	176
Reproduce it yourself	177
The report, verbatim	177
What is worth noticing in that output	178
Honest caveats	178
See also	178

Part I -- Getting started

drag-lint

An AST-exact index, linter and documentation engine for Delphi / Object Pascal.

drag-lint parses your source with tree-sitter, stores symbols, references and call edges in SQLite, and answers questions about them: *where is this declared, who calls it, what breaks if I change it, what is undocumented, what is wrong with it*. It ships as a command-line tool, a RAD Studio IDE plugin, and a language server.

Status: alpha. Expect breaking changes. The index format is stable within a schema version; the CLI surface is not yet frozen.

Start here

Page	For
Features	Everything it does, grouped
Feature Index	Every feature as its own page, by the surface it is reached from
Installation	Getting the CLI, the IDE plugin, or the LSP running
Maintenance	Indexes, the manifest, reindexing, and what to do when something looks wrong
IDE Menu Reference	What every item in the <code>drag-lint</code> menu does
About and Status	The plugin's status window -- versions, connection health, and which indexes are actually in use

The one-paragraph model

Everything drag-lint knows lives in a **SQLite index**. One index per project, stored beside the project file at `<project folder>\_D-RAG\<projectname>.sqlite`, plus one **per-platform library index** covering the RTL, VCL and third-party units. A **manifest** (`drag-lint.json`, beside the exe) lists the sections and lets every command find the right database without being told. If an answer looks wrong, the first question is almost always *is the index fresh?* -- see [Maintenance](#).

Two commands that answer most questions

```
drag-lint query --name TMyClass --db <index>
drag-lint query find-callers --name DoStuff --db <index>
```

and one that finds problems:

```
drag-lint lint-all --db <index>
```

A trap worth knowing on day one

On Windows, if `NoDefaultCurrentDirectoryInExePath` is set (it is, in many corporate images), then

```
cd C:\tools\drag-lint
drag-lint lint-all --db ...      <-- may run a DIFFERENT drag-lint.exe from PATH
```

does **not** run the exe in that folder. Always use `.\drag-lint.exe` or a full path. This is not hypothetical: it once produced 33,626 findings against the real 14,764, silently, because an old binary answered.

Links

- [Issues](#)
- [Releases](#)
- `CHANGELOG.md` in the repository root

Installation

drag-lint has three faces, and they install independently. Most people want the first two.

- 1.
- 2.
- 3.

1. CLI

- 1.

[Releases](#). Win64 is the supported target; a Win32 build exists but is a frozen fallback.

- 1.

```
drag-lint.exe           the tool
tree-sitter-delphi13.dll } parser DLLs -- MUST sit next to the exe
tree-sitter-dfm.dll     }
tree-sitter.dll          }
rules\                  external lint rules (*.scm + *.json)
  builtin-symbols.txt
```

- 1.

Two things that break a fresh install

rules\ must sit beside the exe. The .scm rules load from <exe-dir>\rules. Move the exe without the folder and the external rules silently stop loading -- the built-in checks still run, so you get *fewer* findings rather than an error. Use `--rules-dir <path>` if you must separate them.

The tree-sitter DLLs must be the matching bitness. A Win64 exe that finds Win32 parser DLLs on PATH dies at process start with 0xC000007B (STATUS_INVALID_IMAGE_FORMAT). That is a bitness mismatch, not a missing DLL (0xC0000135), which is why it does not look like a staging problem.

Verify

```
.\drag-lint.exe --version
.\drag-lint.exe rules           # the full, always-current rule catalogue
```

2. Build your first index

The linter works on a bare `.pas` file, but navigation, callers, impact analysis and documentation all need an index.

```
.\drag-lint.exe index C:\path\to\project --db myapp.sqlite  
.\drag-lint.exe query --name TMyClass --db myapp.sqlite
```

For real use, prefer the **manifest** so commands find databases themselves -- see [Maintenance](#).

3. RAD Studio plugin

The plugin is a design-time package, `dclDragLintWizard.bpl`. It adds a top-level **drag-lint** menu to the IDE menu bar (falling back to *Tools* if the main menu is unavailable) and two dockable windows under *View > Tool Windows*.

Install

1.

the IDE holds it.

1.

choice.

1.

`dclDragLintWizard.bpl`.

1.

It needs the CLI

The plugin does not contain the engine -- it **spawns** `drag-lint.exe`. If the menu items fail, check that the exe is reachable and still has `rules\` beside it. `drag-lint > Open Plugin Log` shows what the plugin actually invoked.

Bitness

The IDE is 32-bit, but the plugin spawns the **Win64** CLI by default: the 64-bit engine handles large indexes that the 32-bit one cannot (it runs out of address space on multi-gigabyte databases). The BPL is the only 32-bit artifact.

4. Editors (LSP)

drag-lint ships a stdio language server:

```
drag-lint lsp
```

It provides hover, go-to-definition, **find-references**, **workspace symbols**, completion and signature help from the index -- across every project in the manifest at once. Find-references and workspace-symbols are not duplicated by DelphiLSP, which does not implement them.

- **VS Code** -- extension included at `editors\vscode\drag-lint\`.
- **Neovim / Helix / anything else** -- point the client at `drag-lint lsp` over

stdio.

- **Zed** -- syntax highlighting works today via the tree-sitter grammars;

registering the language server needs a small Rust/WASM extension that is specified but **not built**.

Full settings and the Rust extension spec: `docs/EDITORS.md` in the repository.

Suppressing a finding

A line comment on the offending line:

```
Q.SQL.Text := 'SELECT * FROM t WHERE id=' + Id; // drag-lint:ignore sql-injection-concat
X := X;                                         // drag-lint:ignore    <- all rules
```

For a finding a human has *reviewed and accepted*, prefer the reviewed marker, which is self-invalidating -- it carries a hash of the line's code tokens and re-reports itself if the code changes:

```
except // dl:ok bare-except@7f3a -- rethrown by the caller
```

Reindentation and case changes do **not** invalidate it; a real edit does.

Maintenance

Almost every "drag-lint gave me a wrong answer" turns out to be a stale or mis-scoped index. This page is about keeping that from happening, and about recognising it when it does.

Where indexes live

One database per project, beside the project file:

```
<folder containing the .dproj>\_D-RAG\<project file base name>.sqlite
```

Named after the **project file**, not the folder -- one folder can host several projects.

One database per platform for the library (RTL, VCL, DevExpress, other third-party). These have no owning project folder, so they live together:

```
C:\Projects\.drag-lint\library-Win64.sqlite  
C:\Projects\.drag-lint\library-Win32.sqlite
```

Never guess a path -- ask the tool

```
drag-lint resolve-dbs --platform win64  
drag-lint resolve-dbs --project C:\path\MyApp.dproj  
drag-lint resolve-dbs --in      C:\path\SomeUnit.pas
```

The manifest

`drag-lint.json`, beside the exe, lists every index as a **section** and lets consumers (`query`, `lint-all`, `lsp`, `serve`, `graph`) pick the right database with no `--db`.

A section's **type is declared by its target**: an `include` entry ending in `.dpr/.dproj` makes a **project** section; a folder makes a **library** or folder section. The **mode is chosen per run**: `--rebuild` (from scratch) or `--recompile` (incremental, the default).

A **project index is exactly the compile closure** -- the `.dproj` members, the units they transitively use, each unit's sibling `.dfm`, `{I}` includes, and the project file. Units on the Delphi Library/Browsing path are excluded (they belong to the library index), and loose unreferenced files in the project folder are excluded.

Which databases may answer a question

The authoritative set is the platform library index plus the one project index -- and nothing else.

Another project's database is noise: a name match there is not a caller here. Passing extra databases is how unrelated units once got written into a project's documentation.

Authority is also **per question**: the library index is right for "*which unit declares X*" and wrong for "*who calls X*".

Routine reindexing

```
drag-lint index --all --only <SectionName>      # one section, incremental
drag-lint index --all --jobs 0                  # everything
drag-lint index --all --dry-run                 # preview the plan
```

Reindex a project section by name. Do not point a folder scan at a project database (`index <dir> --db <projectdb>`) -- a project index is a compile closure, and a folder walk changes what the section owns.

Reindex after a build that changed symbols, incrementally. Do not full-rescan the tree after every build.

After an upgrade

The index records an **indexer fingerprint** -- engine version, schema, whether preprocessing was on, and the effective platform. When it changes, every file in scope is re-parsed, because a newer engine may extract symbols the stored parse missed. This is deliberate: the alternative is silently stale parses.

Consequence: **a release upgrade re-parses every index once**. For a large library index that is a long walk. Two things make it survivable:

- **Per-file resume** -- an interrupted re-parse continues where it stopped

instead of restarting.

- **The whole-database announce** -- a long call-target resolve now says so, with

its reason, *before* it runs.

Reading a long run

Set `DRAGLINT_PROFILE=1` for a per-phase breakdown on **stderr**.

```
set DRAGLINT_PROFILE=1
cd C:\tools\drag-lint
.\drag-lint.exe lint-all --db <index> --quiet
```

If you see this, the run is **working, not hung**:

```
resolve: calls  starting WHOLE-DB pass over all 6993 indexed file(s)
resolve: calls  ... whole database because this run rewrote more than one file
                  in three (236 changed, limit 235)
resolve: calls  ... this is the expensive shape (~37 min on a 2 GB index)
                  -- it is running, not hung
```

The call-target resolve has two shapes: a **scoped** pass that re-resolves only what changed, and a **whole-database** pass. The scoped one is several times faster; the message names the reason the scoped one was declined.

Troubleshooting

"database is locked"

An orphaned `drag-lint.exe` (or `drag_lint_graph.exe`) still holds it. **Check first, before diagnosing anything else:**

```
Get-Process drag-lint
```

Note that piping a run through `Select-Object -First N` does **not** kill the process -- it detaches and keeps indexing, and the lock then looks like a broken tool. The IDE plugin also holds a handle on the project's database for the whole session.

An answer is missing a symbol you can see in the source

The index is stale. **Reindex that section and retry** rather than concluding the tool cannot find it.

```
drag-lint index --all --only <Section>
```

A database several engine versions old will answer from an older parser -- quietly, and with fewer results rather than an error.

Output appears only when the run finishes

That is the shell, not the engine. **PowerShell holds a native process's stderr until the process exits.** For long runs redirect through `cmd.exe`, or use `Start-Process -RedirectStandardError`, and poll the file.

The wrong binary answered

If a before/after comparison shows an implausible change, check *which exe ran*. With `NoDefaultCurrentDirectoryInExePath` set, a bare `drag-lint ...` after a `cd` resolves from `PATH`, not the current folder. Use `.\drag-lint.exe`. The quickest tell is the `DRAGLINT_PROFILE` breakdown **format**, which changes whenever the profiler does -- an old-format report means an old binary.

A batch `index --all` reported success but one index is missing

Section builds are independent; a section that fails prints `ERROR building section <name>` and the sweep continues. **Read the log for ERROR, do not trust the exit code alone.** If you hit an intermittent `FOREIGN KEY constraint failed`, keep the log and the database rather than re-running -- a re-run usually succeeds, which is why it stays undiagnosed.

Third-party library updates

Updating a large third-party suite (DevExpress and similar) changes many library units at once, so the library index goes stale and needs a reindex:

```
drag-lint index --all --only Library --jobs 0
```

Expect a whole-database call-target resolve: new units bring new type names, which is exactly the condition that declines the scoped pass. It will announce itself with its reason. Budget accordingly, and let it finish.

Features

Everything drag-lint does, grouped. Counts on this page were taken from the running build (`drag-lint rules --json`), not written by hand.

Three surfaces expose these: the **CLI** (`drag-lint <verb>`), the **RAD Studio plugin** (see [IDE Menu Reference](#)), and the **language server** (`drag-lint lsp`).

Inside the plugin, not everything lives on the main menu. **Auto-fix** and **"allow this finding"** are **reachable only by right-clicking a node in the Structure form** -- see [Fix it](#), [Fix all in unit](#), [Fix all in project](#) and [Allow this message](#). There is also a Project Manager right-click item, [drag-lint: Project Rules...](#).

Indexing

The foundation. Everything below that says *(index)* reads what this produces.

Feature	Command
Index a folder tree	<code>index <path></code>
Index a project's compile closure -- members, transitively used units, sibling <code>.dfm</code> , <code>{I}</code> includes	<code>index --project <file.dproj></code>
Index the Delphi Library + Browsing paths for a platform	<code>index --scan-libraries-win, --scan-libraries-all</code>
Index everything in the manifest, optionally one section	<code>index --all [--only <Sections>]</code>
Incremental (default) vs full rebuild	<code>--recompile / --rebuild</code>
Re-parse everything after an engine upgrade	<code>--force-reparse</code>
Prune + evict files that left the disk or the scope	automatic; <code>--no-prune</code> for a dry look
Watch mode	<code>--watch [--interval N]</code>
Parallel walking	<code>--jobs <n></code>
Cross-index resolution -- consult another index for calls this one cannot resolve	<code>--library-db <lib.sqlite></code>
Per-file resume -- an interrupted walk continues where it stopped	automatic
Preprocessing -- per-config <code>{IFDEF}</code> resolution before parsing	on by default; <code>--no-preprocess</code>
Show which databases a target resolves to	<code>resolve-dbs</code>
Schema inspection / migration	schema , migrate-dbs

Search and navigation (*index*)

Feature	Command
Find a symbol by name or qualified name	<code>query --name / --qname</code>
Text search over string literals -- string constants, resourcestrings, DFM captions, SQL exception messages	<code>query --text "<phrase>"</code>
Who calls this	query find-callers (<code>--resolved</code> for precise call edges)
What does this call	find-callees
Class / interface ancestry, transitively	query ancestors
Resolve a type category (class, interface, float, string, ...)	query typecat
Type of the expression at a cursor position	<code>typeat <file>:<line>:<col></code>
Find by documentation state	<code>query find --doc-tag / --no-docs</code>
Class helpers of a type	helpers-of
Shortest call path between two symbols	<code>call-path --from --to</code>
Symbol slice, class surface, context bundle	<code>slice</code> , <code>surface</code> , context
Hover card	<code>hover</code>
Which unit declares this symbol (and add it to <code>uses</code>)	find-unit

Linting

173 rules. 22 have an auto-fix. 149 are on by default. 119 are built-in checks; 54 are external tree-sitter `.scm` rules you can read and extend in `rules\`.

Run `drag-lint rules` for the always-current catalogue, or `drag-lint rules --category <name>`.

Category	Rules	With auto-fix
bug-patterns	51	5
dead-code	12	6
complexity	11	-
refactoring	11	-
naming	10	8
platform	10	-
project-wide	10	-

Category	Rules	With auto-fix
security	10	-
data-flow	9	-
metrics	8	-
resource-lifetime	8	1
structure	7	-
other	6	-
documentation	5	2
firedac	3	-
review-markers	2	-
Total	173	22

Scopes:

Feature	Command
One file (no index needed)	<code>lint <path></code>
A whole project -- adds project-wide rules, class metrics, duplicate code, documentation drift	<code>lint-all</code>
Project-wide rules only	lint-project
Restrict the report to one project's compile closure	<code>lint-all --project <.dproj></code>
Machine-readable output	<code>--json</code> , and SARIF
Apply the fixable subset	<code>lint-all --fix</code>

Formatting: drag-lint drives **YADF** (the Delphi formatter) for the current unit or the whole active project, straight from the IDE menu -- see [Format with YADF](#) and [Format Whole Project with YADF](#). **Formatting does not invalidate suppressions**. A `dl:ok` hash is computed over a normalised line -- whitespace stripped, identifiers lowercased -- so reindenting and re-spacing cannot change it. Verified on a real file: 10 markers in, 10 markers out, whitespace the only difference.

Suppression:

- `// drag-lint:ignore [rule-id ...] --` silence a line.
- `// dl:ok <rule-id>@<hash> -- reason --` a **reviewed marker**: it carries a

hash of the line's code tokens and **re-reports itself if the code changes**, so a review cannot outlive the code it reviewed. Reindentation and case changes do not invalidate it.

Documentation

DocInsight (/// XML) comments, generated from the index rather than from guesswork.

Feature	Command
Document one symbol, a unit, a project, or everything	document --qname / --unit / --project, document-all
Stubs only, or fully populated facts	--stubs
Managed fact blocks -- callers, callees, used-in-units, raises, returns, wiring, see-also	automatic
Detect documentation that no longer matches the code	doc-drift , and the doc-drift lint rule
Strip generated blocks	document --unit --strip
Generate a doc comment for a symbol	generate-docs
Shared-unit markers, so several projects can document one unit without fighting	shared-unit

Refactoring and code generation

Feature	Command
Rename a symbol or a parameter, index-wide	rename
Extract a method	extract-method
Safe delete (refuses when still referenced)	safe-delete
Add the missing unit for an undeclared identifier	find-unit --apply
Audit and fix <code>uses</code> clauses -- interface->implementation moves, unused units, compiler-verified	uses-audit, uses-fix
Reconcile project members against disk	reconcile-project
Generate an enum helper (ToString, parse, ...)	create-enum-helper
Generate a DUnitX / DUnit test stub	generate-test
Record a reviewed finding	allow

Component conversion

Rule-driven migration of legacy component types (for example Orpheus T0vc* to DevExpress cx/dx).

Feature	Command
Property/event assignability engine over the type tree	proptree
Scaffold a conversion rule from a real from/to pair	convert-scaffold
Validate a rulebook	convert-validate
Apply rules to a unit and its DFM	convert-apply
Visual rulebook editor	ConvRulesEditor.exe

Graphs and reports

Feature	Command	
Call graph, either direction, to a depth	callgraph , reverse-calltree	
Butterfly view (callers + callees of one symbol)	butterfly	
Dependency report	deps-report , uses-report	
Uses cycles -- see the worked example (tool-generated report on a real 4-unit cycle)	<code>cycles</code>	
Impact / blast radius of a change	<code>impact</code>	
Dead code	<code>find-deadcode</code>	
Top symbols by fan-in	<code>top</code>	
TODO / FIXME scan	<code>todos</code>	
Spring4D DI + DFM event wiring	<code>wiring</code>	
Form hierarchy CSV for test helpers	<code>forms-csv</code>	
Export to DOT / Mermaid / Obsidian / Delphi consts	<code>export, --format dot\</code>	mermaid`
Interactive graph viewer	<code>drag_lint_graph.exe</code> , dockable in the IDE	

Compiler integration

Feature	Command
Compile a project or unit and fold the errors into findings	compile-check , check-unit
Compile the unsaved editor buffer ("ghost check")	ghost-check ; IDE: Compile Buffer (unsaved)
Recover files left behind by an interrupted buffer-compile	ghost-recover
Refresh stored compiler findings across a project	<code>refresh-findings</code>
Import an external build log	<code>import-log</code>

Feature	Command
Preprocessor profile for a project/config	pp-profile , preprocess-file

Database and Firebird

Feature	Command
Index .sql migration scripts, including CREATE EXCEPTION messages	index
Snapshot a live Firebird schema into an index	fb-snapshot
Link ORM classes to tables and fields to columns	link-orm

Editor integration

Feature	How
RAD Studio plugin -- 66 entry points across four surfaces, hover, dockable panel and graph	dclDragLintWizard.bpl; see IDE Menu Reference
Language server over stdio -- hover, go-to-definition, find-references , workspace symbols , completion, signature help	drag-lint lsp
VS Code extension	editors\vscode\drag-lint\
Neovim / Helix / any LSP client	point it at drag-lint lsp
HTTP/MCP server for agents	serve
Multi-project workspaces	workspace add / index / status

Maintenance and diagnostics

Feature	Command
Which databases cover what	resolve-dbs, info
Library path drift after a third-party update	library-drift
Per-phase timing breakdown	DRAGLINT_PROFILE=1
Ambiguous call diagnosis	ambiguous-calls
Raw dumps for debugging	dump-refs , dump-call-edges
Index schema report	schema

Counts verified against v1.5.0-alpha. drag-lint ruLes is always the authority -- this page can lag the catalogue.

Part II -- The RAD Studio plugin

IDE Menu Reference

The plugin adds a top-level **drag-lint** menu to the RAD Studio menu bar (it falls back to a submenu under *Tools* if the main menu is unavailable), plus two entries under *View > Tool Windows*.

Every item spawns the `drag-lint.exe` CLI. If an item does nothing, start with **About > Open Plugin Log** - it records what was actually invoked. If the menu's own caption reads **drag-lint (!)**, the LSP server is down; open **About** and read the Connections group for the reason.

Items marked **(index)** need a current index for the active project. If results look thin or stale, see [Maintenance](#).

Pinned at the top

Item	What it does
Full Compile Sweep	Recompiles all units, refreshes the stored compiler findings, then refreshes the open file. The most-used action, which is why it sits above everything else.

Panels

Item	What it does
drag-lint Panel (dockable)	Opens the main dockable tool window. Also under <i>View > Tool Windows > drag-lint</i> .
drag-lint Graph (dockable)	Opens the graph viewer as a dockable window, so it can sit beside Structure. Also under <i>View > Tool Windows > drag-lint Graph</i> . Requires <code>drag_lint_graph.exe</code> deployed beside the BPL.

Everyday actions

Item	What it does
Hover at Cursor (index)	The hover card for the symbol under the caret: signature, documentation, callers.
Go to Definition (index)	Jumps to the declaration of the symbol under the caret.
Show Completion (index)	Index-backed completion list at the caret.
Show Signature Help (index)	Parameter help for the call being typed.
Find Usages... (index)	All references to the symbol under the caret.
Symbol Search... (index)	Search symbols by name across the indexed projects.

Item	What it does
Show Structure	Structural outline of the current unit.
Rename Symbol... <i>(index)</i>	Index-backed rename across the project. Review the preview: a rename is a source-wide edit.
Format with YADF	Formats the current unit with YADF.
Format Whole Project with YADF...	Formats every unit in the project.
Generate Test Helper CSV...	Exports a CSV used by the form/test helper tooling.
drag-lint Options...	The plugin's own options page (rule enablement, profiles, paths).

Uses & Dependencies

Everything about what a unit depends on and what depends on it.

Item	What it does
Circular Uses Report (cycles + fix plan)...	Finds <code>uses</code> cycles and proposes an order of moves that breaks them.
Uses Audit -- interface->impl moves + unused (this unit)...	For the current unit: which <code>uses</code> entries could move from <code>interface</code> to <code>implementation</code> , and which are unused.
Uses Cleanup Preview (compiler-verified, this unit)...	The removals from the audit, verified by compiling , so a unit needed only for an inline or a <code>{ \$IF }</code> branch is not stripped.
Reconcile Project Members (.dpr/.dproj)...	Compares the project file's member list against what is on disk and in the closure.
Uses Report (CSV)...	The dependency data as CSV.
Quick-Fix: Add Unit for Undeclared at Cursor (Ctrl+Alt+U)	The identifier under the caret is undeclared: finds which unit declares it and adds that unit to uses.
Quick-Fix: Add Unit for Inline Hint (H2443) at Cursor	Same, for the inline-expansion hint -- an inline routine whose unit is not in scope.
Quick-Fix: Convert Public Field to Property at Cursor	Rewrites a public field as a property.
Add Missing Units to uses (whole unit)...	The whole-unit form: every unresolved name at once.
Impact / Blast Radius (symbol)... <i>(index)</i>	What is affected if this symbol changes -- the set to retest.
Show Wiring (Spring4D DI + DFM events)... <i>(index)</i>	Bindings the compiler does not make obvious: Spring4D container registrations and DFM event hookups.

Item	What it does
Reverse Call Tree (who calls this, N-deep)... <i>(index)</i>	Callers, transitively, to a chosen depth.
Reverse Call Tree (clickable, Messages window)... <i>(index)</i>	The same tree in the IDE Messages window, so each line navigates.
Call Graph (Butterfly)... <i>(index)</i>	Callers and callees of one symbol together -- the butterfly view.

Inspect Symbol

Item	What it does
Class Surface... <i>(index)</i>	The public surface of a class: members and signatures, without the bodies.
Symbol Slice... <i>(index)</i>	The slice of code relevant to one symbol -- declaration, body, and its immediate context.
Type at Cursor <i>(index)</i>	Resolves the static type of the expression under the caret.

Code Quality

Item	What it does
Run Lint All (Full Report)... <i>(index)</i>	The full project lint: .scm rules, built-in AST checks, project-wide rules, class metrics, duplicate code and documentation drift. Writes a report file.
Find Dead Code... <i>(index)</i>	Symbols nothing references.
Find Undocumented (public)... <i>(index)</i>	Public declarations with no documentation comment.
Scan TODOs / FIXMEs...	TODO/FIXME/HACK comments across the project.
Compiler Hints...	The stored compiler findings, from the last sweep or an imported log.
Top Symbols (fan-in)... <i>(index)</i>	The most-depended-on symbols -- where a change ripples furthest.

Generate && Export

Item	What it does
Doc Comment Stub (symbol)... <i>(index)</i>	A DocInsight comment skeleton for one symbol, with index-grounded facts.
Auto-Document Whole Project... <i>(index)</i>	Generates or refreshes managed documentation blocks across the project. Writes to source files -- commit or stash first.
Unit Test Stub (symbol)...	A test skeleton for the symbol.

Item	What it does
Export Enums (Delphi const)...	Enumerations as Delphi constant declarations.
Export Graph (DOT)... (<i>index</i>)	The dependency/call graph in Graphviz DOT.
Export to Obsidian... (<i>index</i>)	Exports index knowledge as Obsidian-style linked notes.

Index && Maintenance

Item	What it does
Rebuild Index for This Project	Destructive by design. Clears this project's index and re-parses its whole compile closure. Named "Rebuild", not "Reindex", so it cannot be confused with an incremental refresh.
Show Resolved DBs (debug)...	Which databases this project resolves to. The first thing to check when results come from somewhere unexpected.
Library Drift Check...	Compares the library index against the current Library/Browsing paths and reports what has moved -- run it after a third-party suite update.

Compile && Analysis

Below a separator, under a section header. Daily actions, which is why they sit on the menu rather than behind the About window.

Item	What it does
Compile && Diagnose	Compiles, then reports diagnostics together with compiler output.
Compile Buffer (unsaved)	Compiles the unsaved buffer -- the "ghost check", for errors that only exist in what you are typing.

About

Item	What it does
About	Opens the About and Status window -- versions, connection health, which indexes are actually in use, configuration warnings, process footprint, and the diagnostic actions listed below.

The seven items that used to sit under a *Diagnostics && Tests* section header are **no longer on the menu**. They are buttons on the About window, because each one is reached when something is already wrong -- which is exactly when the rest of that window's information is wanted too:

Moved action	Now at
Run Diagnostics (didSave)	About > Run Diagnostics (didSave)
Run AST Checks	About > Run AST Checks
Lint Buffer (Unsaved)	About > Lint Buffer (Unsaved)
Copy Diagnostics (Current File)	About > Copy Diagnostics (Current File)
Recover Buffer-Compile Files	About > Recover Buffer-Compile Files
Import Build Log...	About > Import Build Log...
Open Plugin Log	About > Open Plugin Log

View > Tool Windows

Item	What it does
drag-lint	The dockable panel.
drag-lint Graph	The dockable graph viewer.

Both are also reachable from the top of the `drag-lint` menu. The plugin removes any stale entries of the same name on load, so an earlier install cannot leave a dead item that calls into an unloaded package.

About and Status

drag-lint > About

The plugin's status window: what versions are running, whether each component is reachable, **which index files are actually being used**, which settings are misconfigured, and how much memory the IDE is holding. It also hosts the diagnostic actions that used to live under a *Diagnostics & Tests* menu section.

Why it exists

For several months the IDE answered hover, go-to-definition and completion from a 1.5 GB library index left over from before the 2026-08-11 layout change. Nothing appeared broken, because a stale index answers confidently -- just with older and fewer results. Rebuilding the library indexes changed nothing, which made it look like a rebuild problem rather than a resolution problem.

The cause was a manifest lookup built from `Settings.ExePath`, which defaults to the bare name `drag-lint.exe`. `ExtractFilePath` of that is empty, so the plugin looked for a *relative* `drag-lint.json` against the IDE's working directory, found none, and fell back -- silently, through four separate exits that recorded nothing.

One line of UI would have exposed it in seconds instead of a day. That line is now the first thing in the Indexes group.

The groups

Group	What it tells you
Versions	Plugin version and BPL build time, engine version and build date, tree-sitter grammar versions, whether the graph viewer is deployed.
Connections	LSP state (connected / starting / DOWN, with the reason), whether the engine exe was found, and whether every resolved index exists on disk.
Indexes in use	The active project and platform, then the library index path actually in use, its size and write date, and the rule that chose it . Then every other resolved index.
Configuration	Settings that misconfigure the plugin without any visible symptom -- a bare-name <code>ExePath</code> , a <code>DbPathTemplate</code> predating the <code>_D-RAG</code> layout, a disabled library index, a missing manifest.
Process	Free system RAM, the IDE's private bytes and handle count. The IDE is a 32-bit process, so these are real operating limits rather than trivia.

Values are coloured: **green** healthy, **amber** suspicious, **red** broken. A red line is the one to act on.

Reading the library line

The most important line in the window is the rule that chose the library index:

- `manifest <path> -> outDir <dir>, platform Win32 -- trustworthy`. The

index was selected from the manifest for the active project's platform.

- `FALLBACK -- ...` -- shown in red. The manifest rule declined, and the

message says exactly why (no manifest at that path, no `indexes.outDir`, no `library-<platform>.sqlite` in the `outDir`, or a parse failure). A fallback is treated as a defect rather than a lesser mode, because a fallback index may be from any era.

Actions

Button	What it does
Diagnose Current State	Builds the full text report and shows it in a copyable window. This is what to paste into an issue.
Copy Report	The same report, straight to the clipboard.
Refresh	Re-reads every group.
Open Plugin Log	The plugin's own log -- start here when a menu item misbehaves.
Run Diagnostics (didSave)	The diagnostics pass the editor integration fires on save.
Run AST Checks	The built-in AST checks only, on the current file.
Lint Buffer (Unsaved)	Lints the editor buffer, including unsaved edits.
Copy Diagnostics (Current File)	The current file's findings, to the clipboard.
Recover Buffer-Compile Files	Recovers temporary files an interrupted buffer compile left behind.
Import Build Log...	Loads an external build log so its errors become browsable findings.

Performance

Opening this window does **not** open a database. Index facts come from the file system only -- path, size, write time.

This is a deliberate constraint, not an optimisation. On the 3 GB `library-Win32.sqlite`, `schema --format json` takes **38 seconds**, because it runs `COUNT(*)` on every table; opening the same file and doing an indexed lookup takes **0.5 seconds**. A status screen that is slow does not get opened, and a status screen nobody opens reports nothing.

(That 38-second figure also caused a wrong diagnosis once: it was read as "a large index is slow to open" and produced a filed defect aimed at the wrong target. The real cost of opening a multi-GB index is half a second.)

The connection indicator

When the LSP server is down the top-level menu caption becomes **drag-lint (!)** and stays that way while the condition holds. The plugin retries the connection after 30 seconds, lazily -- on the next action that needs the server, not on a timer.

There is deliberately no modal dialog. The previous one announced "*LSP initialize handshake failed*" for a handshake that had actually **succeeded**, 56 seconds after the client stopped waiting -- and that sentence sent a day of debugging at the server when the real cause was the plugin's own process storm saturating the machine. The window shows the recorded error verbatim instead of paraphrasing it into a claim.

See also

- [IDE Menu Reference](#)
- [Maintenance](#)

Add Missing Units to uses (whole unit)

The whole-unit form of the undeclared-identifier quick-fix: resolves every unresolved name in a unit at once and adds the units that declare them. Reach for it after a large paste or a big refactor leaves several names unresolved.

Running it from the CLI

```
drag-lint check-unit <unit.pas> [--project <dproj>] [--platform win32|win64] [--shadow <dir>] [--resolve-uses] [--db PATH] [--format json|text]
```

`--resolve-uses` is what drives the add-missing-units behaviour; `--project` and `--platform` scope the check; `--shadow <dir>` and `--format` are also available.

Reaching it in the IDE

drag-lint > Uses & Dependencies > Add Missing Units to uses (whole unit)...

What it needs

Optional. An index is not marked required, but resolving names to their declaring units depends on what has been indexed.

Example

Illustrative only:

```
drag-lint check-unit C:\Projects\MyApp\CustomerOrder.pas --project  
C:\Projects\MyApp\MyApp.dproj --resolve-uses --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would resolve every unresolved name in `CustomerOrder.pas` and add the units that declare them.

Allow this message

Records a `dl:ok` review of one finding, so it stops being reported without being fixed. Reach for it when a finding is a deliberate, reviewed exception rather than something to change.

Reaching it in the IDE

This is a RIGHT-CLICK action, not a main-menu item. Menu path: "Structure form (right-click a tree node)".

The Structure form's right-click menu is the only place "Allow this message" is reachable in the IDE -- there is no main drag-lint menu item for it.

To reach it: open the Structure form first (drag-lint > Show Structure), then right-click the tree node for a finding and choose "Allow this message".

Running it from the CLI

The feature map lists the underlying verb as `allow`. From the usage banner:

```
drag-lint allow <file> --fix-line <L> --fix-rule <id> [--apply]
```

Records a `dl:ok` review of one finding; dry-run without `--apply`.

What it needs

No index required -- `allow` records the review directly against the given file, without going through a project index database.

Example

Illustrative:

```
drag-lint allow C:\Projects\MyApp\Unit1.pas --fix-line 42 --fix-rule unused-private-member --apply
```

This would record `Unit1.pas` line 42's `unused-private-member` finding as reviewed, so it no longer appears in future lint runs.

Auto-Document Whole Project

Writes DocInsight documentation into every public declaration a project owns. Reach for it to bulk-document a project rather than symbol by symbol.

Running it from the CLI

This runs as a 3-step batch, not a single command:

```
drag-lint index --project <file.dproj> --db <db>
drag-lint document --project <file.dproj> --apply --db <db>
drag-lint index --project <file.dproj> --db <db>
```

Step 1 freshens the index. Step 2 writes documentation into every public declaration the project owns (-apply commits the edits; without it the command only previews). Step 3 re-indexes: --apply shifts line numbers in the files it edited, and the index has to reflect the new positions before it is trusted again.

document --project also accepts --stubs, --json, --no-backup, --include-accessors, --reindex (self-freshens the index after applying, folding steps 2-3 into one call), and --document-third-party.

Reaching it in the IDE

drag-lint > Generate & Export > Auto-Document Whole Project...

What it needs

An active project. Unusually for this tool, **no pre-existing index is required**: step 1 of the batch is an index run, and indexing creates the database if it is missing and re-parses only stale or absent files. So this action is safe to run on a project that has never been indexed -- it will simply do the indexing first.

The --db flag is optional when you run document yourself -- drag-lint auto-resolves the project's index when it is omitted.

Example

Illustrative only:

```
drag-lint document --project C:\Projects\MyApp\MyApp.dproj --apply --db
C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would document every public declaration MyApp.dproj owns, followed by a re-index of the same project and DB.

Call Graph (Butterfly)

Shows callers and callees of one symbol together in a single "butterfly" view. Reach for it to see both directions of a symbol's call graph at once.

Running it from the CLI

There are two ways to get this chart, and they are genuinely different -- this is a deliberate asymmetry, not a duplicate.

The `butterfly` verb composes both wings in ONE command:

```
drag-lint butterfly --qname <X> [--depth N] [--format dot|mermaid|text|json] [--output F]
--db PATH [--db ...]
```

The IDE menu item does NOT call `butterfly`. It runs `reverse-calltree` TWICE -- once for callers, once with `--direction callees` -- and composes the two JSON documents itself into the dock's Call Graph tab:

```
drag-lint reverse-calltree --qname <X> --db <db> --depth 3 --format json
drag-lint reverse-calltree --qname <X> --db <db> --depth 3 --format json --direction
callees
```

So `butterfly` is CLI-only: no IDE action invokes it. If you want the chart from a script, `butterfly` is the single-command option.

Reaching it in the IDE

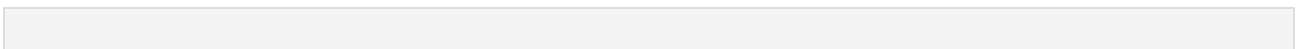
drag-lint > Uses & Dependencies > Call Graph (Butterfly)...

What it needs

An index IS required -- both `butterfly` and the two `reverse-calltree` calls the IDE composes it from read the project's index database. The project's DB lives at `<project folder>\_D-RAG\<project file>.sqlite`; run `drag-lint resolve-dbs --project <X.dproj>` if you are unsure which one.

Example

Illustrative only:



```
drag-lint reverse-calltree --qname CustomerOrder.TCustomerOrder.Total --db  
C:\Projects\MyApp\_D-RAG\MyApp.sqlite --format dot
```

This would produce a DOT-format graph touching the `Total` member of `TCustomerOrder`.

Circular Uses Report (cycles + fix plan)

Finds `uses` cycles among indexed units and, on request, proposes a followable refactoring plan to break them. Reach for it when the compiler is fighting circular references, or a project's dependency graph feels tangled.

Running it from the CLI

```
drag-lint cycles --db <file.sqlite> [--edges] [--causes] [--plan] [--format json|text]
```

`--edges` and `--causes` add detail to the cycle report; `--plan` produces the followable refactoring playbook; `--format` selects json or text output.

Reaching it in the IDE

drag-lint > Uses & Dependencies > Circular Uses Report (cycles + fix plan)...

What it needs

Required. You must have indexed the project before running this -- cycle detection reads unit-to-unit `uses` edges from the index, not from disk.

Example

Illustrative only, against a project database that has already been indexed:

```
drag-lint cycles --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --plan --format text
```

This would report any `uses` cycles involving, say, the unit that declares `TfrmMain`, plus a suggested order of moves to break them.

Class Surface...

Shows the public surface of a class: its members and signatures, without the method bodies. Reach for it to see what a class exposes without reading the whole unit.

Reaching it in the IDE

drag-lint > Inspect Symbol > Class Surface...

No keyboard shortcut is documented for this menu item.

Running it from the CLI

```
drag-lint surface --qname <Foo.TBar> [--db <file.sqlite>] [--include-impl] [--all-visibility] [--format text|json]
```

--qname is the fully qualified class; --include-impl adds implementation-only members; --all-visibility includes non-public members; --format selects text or JSON.

What it needs

The --db flag is optional -- drag-lint auto-resolves the index when omitted. An index must still exist.

Example

Illustrative only:

```
drag-lint surface --qname CustomerOrder.TCustomerOrder --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would print the public surface of TCustomerOrder.

Compile Buffer (unsaved)

Compiles a project with the current editor buffer's unsaved content substituted for its file on disk, then restores the original file unchanged. Reach for it to get compiler diagnostics against unsaved edits without saving them first.

Running it from the CLI

The feature map lists this action's CliVerb as "(none - in-process)", but this IDE action actually runs the `ghost-check` verb. Running it bare prints its own usage banner and exits 2:

```
Usage: drag-lint ghost-check <dproj> ( --unit <real.pas> --buffer <buf> | --overlays
<manifest> ) [--platform win32|win64] [--format json|text]
```

`<dproj>` is the project to compile. Give either a single `--unit <real.pas> --buffer <buf>` pair, or an `--overlays <manifest>` file listing multiple real-path/buffer-path pairs. `--platform` selects `win32` or `win64`; `--format` selects `json` or `text`.

Reaching it in the IDE

drag-lint > Compile Buffer (unsaved)

What it needs

The feature map marks this row's Index column "n/a". It needs the active project (the `.dproj`) and the unsaved buffer content for the unit being compiled.

Example

Illustrative only:

```
drag-lint ghost-check C:\Projects\MyApp\MyApp.dproj --unit
C:\Projects\MyApp\frmOrders.pas --buffer C:\Projects\MyApp\_D-RAG\frmOrders.buf --
platform win32 --format json
```

This would compile `MyApp.dproj` with `frmOrders.pas`'s content replaced by the buffer file, report diagnostics as JSON, then restore `frmOrders.pas` unchanged.

Compile & Diagnose

Compiles a project or unit and folds the compiler's errors and warnings into drag-lint findings. Reach for it to get compiler diagnostics for the current file or project without leaving the IDE.

Running it from the CLI

The feature map lists this action's CliVerb as "(none - in-process)", but this IDE action actually runs the `compile-check` verb:

```
drag-lint compile-check <target.dproj|.pas> [--db PATH] [--format json|text]
```

`<target.dproj|.pas>` is the project or unit to compile; `--db` is an index database; `--format` selects `json` or `text` output.

Reaching it in the IDE

drag-lint > Compile & Diagnose

What it needs

The feature map marks this row's Index column "n/a". It needs an active project or the current file to compile, and an open editor to show the resulting diagnostics.

Example

Illustrative only:

```
drag-lint compile-check C:\Projects\MyApp\MyApp.dproj --format json
```

This would compile `MyApp.dproj` and print the resulting diagnostics as JSON.

Compiler Hints

Uses `query hints` to surface compiler hint/warning findings that have been imported into the index, filtered by code or severity. Reach for it to review compiler-reported issues without re-running a build.

Running it from the CLI

```
drag-lint query hints --db <file.sqlite> [--name <code>] [--rule <severity>]
```

`--name` filters by hint code; `--rule` filters by severity.

Reaching it in the IDE

drag-lint > Code Quality > Compiler Hints...

What it needs

An index IS required. The project's DB lives at `<project folder>\_D-RAG\<project file>.sqlite`; run `drag-lint resolve-dbs --project <X.dproj>` if unsure.

Example

Illustrative only:

```
drag-lint query hints --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --rule warning
```

This would list warning-severity compiler hints recorded in MyApp's index.

Copy All Diagnostics

Puts every lint finding for the current unit onto the clipboard, starting from the Structure form. Reach for it when you want to paste a unit's full findings list elsewhere (a chat, an issue, a note) instead of reading them in the IDE.

Reaching it in the IDE

This is a RIGHT-CLICK action, not a main-menu item. Menu path: "Structure form (right-click a tree node)".

To reach it: open the Structure form first (drag-lint > Show Structure), then right-click a tree node and choose "Copy All Diagnostics".

Running it from the CLI

The feature map lists the underlying verb as `lint`. From the usage banner:

```
drag-lint lint <path> [--rule <id>] [--disable id1,id2] [--rules-dir <dir>] [--json]
```

This runs the lint pass on a file and prints its findings. The "copy to clipboard" behavior is an IDE-side convenience on top of this verb; the CLI itself only prints, it does not clipboard-copy.

What it needs

No index required -- `lint <path>` lints the given file directly, without going through a project index database.

Example

Illustrative:

```
drag-lint lint C:\Projects\MyApp\Unit1.pas --json
```

This would print `Unit1.pas`'s findings as JSON; the IDE action copies the equivalent findings text to the clipboard instead.

Copy Diagnostics (Current File)

Puts the current file's lint findings onto the clipboard. Reach for it to paste a file's findings elsewhere (a chat, an issue, a note) instead of reading them in the IDE.

Running it from the CLI

```
drag-lint lint <path> [--rule <id>] [--disable id1,id2] [--rules-dir <dir>] [--json]
```

This runs the lint pass on a file and prints its findings. The clipboard-copy behavior is an IDE-side convenience on top of this verb; the CLI itself only prints.

Reaching it in the IDE

drag-lint > Copy Diagnostics (Current File)

What it needs

No index needed. `lint <path>` lints the given file directly, without going through a project index database; the feature map marks this row's Index column "no".

Example

Illustrative only:

```
drag-lint lint C:\Projects\MyApp\Unit1.pas --json
```

This would print `Unit1.pas`'s findings as JSON; the IDE action copies the equivalent findings text to the clipboard instead.

Create helper class

Generates a Byte-family record helper for an enum type represented by a Structure-tree node. Reach for it when you have an enum and want the usual ToByte/FromByte/ToInteger/FromInteger/ToString/FromString helper methods without writing them by hand.

Reaching it in the IDE

This is a RIGHT-CLICK action, not a main-menu item. Menu path: "Structure form (right-click a tree node)".

To reach it: open the Structure form first (drag-lint > Show Structure), then right-click an enum type's node in the tree and choose "Create helper class".

Running it from the CLI

The feature map lists the underlying verb as `create-enum-helper`. From the usage banner:

```
drag-lint create-enum-helper --qname <TEnum> [--apply|--json|--no-backup] [--methods <csv>] [--tostring rtti|case] [--db PATH]
```

--methods defaults to all six (tobyte,frombyte,tointeger,frominteger, tostring,fromstring). --tostring defaults to `rtti` (GetEnumName) or can be `case` (explicit case statement). Idempotent: if a helper for the enum already exists anywhere in the index, the action reports `exists` and makes no edit.

What it needs

Optional, per the feature map's Index column.

Example

Illustrative:

```
drag-lint create-enum-helper --qname Unit1.TColorKind --apply --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would generate a record helper with all six default methods for `TColorKind`.

Doc Comment Stub (symbol)

Uses `generate-docs` to produce a doc-comment stub for one qualified symbol. Reach for it to start a DocInsight comment on a specific method, type, or property.

Running it from the CLI

```
drag-lint generate-docs --qname <Foo.TBar.Baz> [--format xmlDoc|pasdoc] [--db PATH]
```

`--qname` is the fully qualified symbol; `--format` selects `xmlDoc` or `pasdoc` output.

Reaching it in the IDE

drag-lint > Generate & Export > Doc Comment Stub (symbol)...

What it needs

The `--db` flag is optional -- drag-lint auto-resolves the project's index when omitted. An index must still exist.

Example

Illustrative only:

```
drag-lint generate-docs --qname MyApp.TCustomerOrder.Total --format xmlDoc --db  
C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would produce a doc-comment stub for the `Total` member of `TCustomerOrder`.

Document it

Generates or repairs a managed DocInsight comment on the single symbol represented by a Structure-tree node. Reach for it when one declaration needs a doc comment and you do not want to touch the rest of the unit.

Reaching it in the IDE

This is a RIGHT-CLICK action, not a main-menu item. Menu path: "Structure form (right-click a tree node)".

To reach it: open the Structure form first (drag-lint > Show Structure), then right-click a node in the tree and choose "Document it".

Running it from the CLI

The feature map lists the underlying verb as `document`. From the usage banner, the single-symbol form:

```
drag-lint document --qname <Foo.TBar.Baz> [--apply|--json|--no-backup] [--db PATH]
```

Generates/repairs a managed DocInsight comment for that one qualified name.

What it needs

Optional, per the feature map's Index column.

Example

Illustrative:

```
drag-lint document --qname Unit1.TMyClass.DoWork --apply --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would write a managed DocInsight comment onto `DoWork`.

Document project

Generates or repairs managed DocInsight comments for every public declaration the project owns, starting from a Structure-tree node's project. Reach for it when a whole project needs documentation coverage, not just one unit or symbol.

Reaching it in the IDE

This is a RIGHT-CLICK action, not a main-menu item. Menu path: "Structure form (right-click a tree node)".

To reach it: open the Structure form first (drag-lint > Show Structure), then right-click a node in the tree and choose "Document project".

Running it from the CLI

The feature map lists the underlying verb as `document`. From the usage banner, the whole-project form:

```
drag-lint document --project <p.dpr|.dproj> [--stubs|--apply|--json|--no-backup|--include-accessors|--reindex|--document-third-party] [--db PATH]
```

Documents every public declaration the project owns (scope: the compile closure restricted to the project's own roots). `--reindex` self-freshens the index afterward so hover/LSP are correct immediately.

What it needs

Optional, per the feature map's Index column.

Example

Illustrative:

```
drag-lint document --project C:\Projects\MyApp\MyApp.dproj --apply --reindex --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would write managed DocInsight comments across the whole project and refresh the index afterward.

Document unit

Generates or repairs managed DocInsight comments for every public declaration in the unit represented by a Structure-tree node (facts-only). Reach for it when a whole unit needs documentation, not just one symbol.

Reaching it in the IDE

This is a RIGHT-CLICK action, not a main-menu item. Menu path: "Structure form (right-click a tree node)".

To reach it: open the Structure form first (drag-lint > Show Structure), then right-click a node in the tree and choose "Document unit".

Running it from the CLI

The feature map lists the underlying verb as `document`. From the usage banner, the whole-unit form:

```
drag-lint document --unit <file.pas> [--apply|--json|--no-backup|--include-accessors] [--db PATH]
```

Documents every public declaration in the unit (facts-only). Trivial Get*/Set* accessors are skipped unless `--include-accessors` is given.

What it needs

Optional, per the feature map's Index column.

Example

Illustrative:

```
drag-lint document --unit C:\Projects\MyApp\Unit1.pas --apply --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would write managed DocInsight comments onto every public declaration in `Unit1.pas`.

drag-lint

This page documents the "View > Tool Windows > drag-lint" menu entry -- the main drag-lint dockable tool window, reached from RAD Studio's standard Tool Windows menu instead of the drag-lint menu. It is the same panel as [drag-lint Panel \(dockable\)](#); see that page for the drag-lint-menu path to it. For the product overview, see the wiki [Home](#) page.

Reaching it in the IDE

View > Tool Windows > drag-lint

No keyboard shortcut is documented for this menu item.

Running it from the CLI

There is no standalone CLI verb for this (feature map CliVerb: "(none - in-process)"). It is served in-process by the IDE plugin.

What it needs

The feature map marks this row's Index column "n/a" and its Mechanism "in-process". It needs the drag-lint plugin loaded in the IDE.

Example

Illustrative: choose "View > Tool Windows > drag-lint" to open the main drag-lint panel.

drag-lint Graph

This page documents the "View > Tool Windows > drag-lint Graph" menu entry -- the drag-lint graph viewer, reached from RAD Studio's standard Tool Windows menu instead of the drag-lint menu. It is the same window as [drag-lint Graph \(dockable\)](#); see that page for the drag-lint-menu path to it.

Reaching it in the IDE

View > Tool Windows > drag-lint Graph

No keyboard shortcut is documented for this menu item.

Running it from the CLI

There is no standalone CLI verb for this (feature map CliVerb: "(none - in-process)"). It is served in-process by the IDE plugin.

What it needs

The feature map marks this row's Index column "n/a" and its Mechanism "in-process". It needs the drag-lint plugin loaded in the IDE, and requires `drag_lint_graph.exe` deployed beside the plugin's BPL (per the [IDE Menu Reference](#)).

Example

Illustrative: choose "View > Tool Windows > drag-lint Graph" to open the graph viewer.

drag-lint Graph (dockable)

Opens the drag-lint graph viewer as a dockable window inside the IDE, so it can sit beside Structure. Reach for it to keep the graph viewer docked instead of a separate window.

Reaching it in the IDE

drag-lint > drag-lint Graph (dockable)

Also reachable from View > Tool Windows > drag-lint Graph -- the same dockable window under a second menu path; see [drag-lint Graph](#) for that entry.

No keyboard shortcut is documented for this menu item.

Running it from the CLI

There is no standalone CLI verb for this (feature map CliVerb: "(none - in-process)"). It is served in-process by the IDE plugin.

What it needs

The feature map marks this row's Index column "n/a" and its Mechanism "in-process". It needs the drag-lint plugin loaded in the IDE, and requires `drag_lint_graph.exe` deployed beside the plugin's BPL (per the [IDE Menu Reference](#)).

Example

Illustrative: choose "drag-lint > drag-lint Graph (dockable)" to dock the graph viewer beside the Structure form.

drag-lint Options

The plugin's settings dialog for drag-lint: lets you browse and toggle the lint rule catalog. Reach for it to see which rules exist, which are on by default, and which have an auto-fix, and to change what is enabled.

Running it from the CLI

The dialog itself is in-process (no CLI shell-out), but it loads its rule catalog by running:

```
drag-lint rules --json
```

The equivalent standalone command, with the same catalog:

```
drag-lint rules [--json] [--category <name>] [--rules-dir <dir>]
```

As loaded by this dialog, the catalog has 173 rules, 22 of them fixable, 149 on by default, across 16 categories.

Reaching it in the IDE

drag-lint > drag-lint Options...

What it needs

Not documented as needing an index -- the feature map marks this row's Index column "n/a" and its Mechanism "in-process". It needs the rule catalog files that `rules --json` reads; it does not need a project database.

Example

Illustrative only: opening drag-lint > drag-lint Options... shows the full 173-rule catalog, grouped by its 16 categories, with a toggle per rule and a marker for which of the 22 fixable rules have an auto-fix.

drag-lint Panel (dockable)

Opens the main drag-lint dockable tool window inside the IDE. Reach for it to keep drag-lint's panel docked alongside your editor instead of a separate window.

Reaching it in the IDE

drag-lint > drag-lint Panel (dockable)

Also reachable from View > Tool Windows > drag-lint -- the same dockable panel under a second menu path; see [drag-lint](#) for that entry.

No keyboard shortcut is documented for this menu item.

Running it from the CLI

There is no standalone CLI verb for this (feature map CliVerb: "(none - in-process)"). It is served in-process by the IDE plugin.

What it needs

The feature map marks this row's Index column "n/a" and its Mechanism "in-process". It needs the drag-lint plugin loaded in the IDE.

Example

Illustrative: choose "drag-lint > drag-lint Panel (dockable)" to dock the main drag-lint panel beside the editor.

drag-lint: Project Rules...

Shows the catalog of lint rules in the context of a specific project. Reach for it when you want to see which rules exist (and their categories) starting from the Project Manager rather than the drag-lint main menu.

Reaching it in the IDE

This is a RIGHT-CLICK action, not a main-menu item. Menu path: "Project Manager (right-click a project)".

Unlike the other IDE-context rows on this wiki, this one is NOT reached through the Structure form. To reach it: in RAD Studio's Project Manager, right-click a project and choose "drag-lint: Project Rules...".

Running it from the CLI

The feature map lists the underlying verb as `rules`. From the usage banner:

```
drag-lint rules [--json] [--category <name>] [--rules-dir <dir>]
```

Lists every lint rule (catalog).

What it needs

No index needed, per the feature map's Index column.

Example

Illustrative:

```
drag-lint rules --category naming --json
```

This would print the catalog of naming-category rules as JSON.

Export Enums (Delphi const)

Uses `export enums` with `--format delphi-const` to dump indexed enum types as Delphi const declarations. Reach for it to generate a const-based mirror of an enum for interop or codegen.

Running it from the CLI

```
drag-lint export enums --db <file.sqlite> [--format firebird-sql|csv|json|delphi-const]
```

`--format` selects the output flavor: `firebird-sql`, `csv`, `json`, or `delphi-const`.

Reaching it in the IDE

drag-lint > Generate & Export > Export Enums (Delphi const)...

What it needs

An index IS required. The project's DB lives at `<project folder>\_D-RAG\<project file>.sqlite`; run `drag-lint resolve-dbs --project <X.dproj>` if unsure.

Example

Illustrative only:

```
drag-lint export enums --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --format delphi-const
```

This would print MyApp's indexed enum types as Delphi const declarations.

Export Graph (DOT)

Uses `graph` to export the indexed call/reference graph in DOT format, either whole or rooted at a name substring. Reach for it to visualize dependency structure with Graphviz.

Running it from the CLI

```
drag-lint graph --db <file.sqlite> [--format dot|mermaid] [--name <root-substr>] [--output <file>]
```

`--format` selects `dot` or `mermaid`; `--name` roots the graph at a substring match; `--output` writes to a file instead of stdout.

Reaching it in the IDE

drag-lint > Generate & Export > Export Graph (DOT)...

What it needs

An index IS required. The project's DB lives at `<project folder>\_D-RAG\<project file>.sqlite`; run `drag-lint resolve-dbs --project <X.dproj>` if unsure.

Example

Illustrative only:

```
drag-lint graph --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --format dot --name TCustomerOrder --output order.dot
```

This would write a DOT graph rooted around `TCustomerOrder` to `order.dot`.

Export to Obsidian

Uses `export obsidian` to write the indexed project out as a set of Obsidian vault markdown pages. Reach for it to build a browsable wiki of a project's symbols.

Running it from the CLI

```
drag-lint export obsidian --db <file.sqlite> --output-dir <dir> [--open]
```

`--output-dir` is required and names the folder to write pages into; `--open` opens it after export.

Reaching it in the IDE

drag-lint > Generate & Export > Export to Obsidian...

What it needs

An index IS required. The project's DB lives at `<project folder>\_D-RAG\<project file>.sqlite`; run `drag-lint resolve-dbs --project <X.dproj>` if unsure.

Example

Illustrative only:

```
drag-lint export obsidian --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --output-dir  
C:\Projects\MyApp-wiki --open
```

This would write MyApp's indexed symbols as markdown pages into `C:\Projects\MyApp-wiki` and open the folder afterward.

Find Dead Code

Scans the index for code with no references -- candidates to delete. Reach for it before a cleanup pass or before removing something you suspect is unused.

Running it from the CLI

```
drag-lint find-deadcode [--kind method|function|...] [--include-private] [--db PATH]
```

--kind filters by symbol kind (for example `method` or `function`); --include-private also considers private members.

Reaching it in the IDE

drag-lint > Code Quality > Find Dead Code...

What it needs

The --db flag is optional -- drag-lint auto-resolves the project's index when omitted. An index must still exist.

Example

Illustrative only:

```
drag-lint find-deadcode --kind method --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would list methods in MyApp's index with no resolved references.

Find Undocumented (public)

Uses `query find` to list indexed symbols filtered by documentation state, kind, or visibility. Reach for it to see which public symbols still have no DocInsight comment.

Running it from the CLI

```
drag-lint query find [--doc-tag X | --doc-contains Y | --no-docs] [--kind K] [--public] [--db ...]
```

`--no-docs` restricts results to symbols with no doc comment; `--public` restricts to public symbols; `--doc-tag/--doc-contains` filter by tag or tag content instead of by absence; `--kind` filters by symbol kind.

`--doc-tag` accepts exactly nine tags: `summary`, `remarks`, `returns`, `param`, `exception`, `example`, `seealso`, `since`, `deprecated`. An unrecognized tag errors (exit 2) and lists the valid set.

Reaching it in the IDE

drag-lint > Code Quality > Find Undocumented (public)...

What it needs

The `--db` flag is optional -- drag-lint auto-resolves the project's index when omitted. An index must still exist.

Example

Illustrative only:

```
drag-lint query find --no-docs --public --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would list MyApp's public symbols that carry no doc comment.

Find Usages...

Lists every reference to the symbol under the editor caret across the indexed project. Reach for it before changing or removing a symbol, to see everywhere it is used.

Reaching it in the IDE

drag-lint > Find Usages...

Keyboard shortcut: Ctrl+Alt+F.

Running it from the CLI

There is no standalone CLI verb for this menu item (feature map CliVerb: "(none - in-process)"). It is served in-process by the IDE plugin. The bundled language server exposes find-references over stdio to other editors -- the [Features](#) page lists find-references among the methods `drag-lint lsp` implements - but that is a stdio protocol server, not a one-shot CLI command.

This is a different feature-map row from the right-click "Find Usages" action on a Structure-tree node, which does have an underlying verb -- see [Find Usages \(context\)](#).

What it needs

The feature map marks this row's Index column "n/a" and its Mechanism "in-process". It needs an open editor with the caret on the target symbol.

Example

Illustrative: with the caret on `TCustomerOrder.Total`, choose "Find Usages..." to list every reference to `Total` across the indexed project.

Find Usages (context)

Lists references to the symbol represented by a Structure-tree node. Reach for it when you are browsing the Structure outline and want the usages of a symbol without first navigating to it in the editor and using the main "Find Usages..." menu item (a separate, in-process feature-map row).

Reaching it in the IDE

This is a RIGHT-CLICK action, not a main-menu item. Menu path: "Structure form (right-click a tree node)".

To reach it: open the Structure form first (drag-lint > Show Structure), then right-click a node in the tree and choose "Find Usages".

Running it from the CLI

The feature map lists the underlying verb as `usages`. See the [usages](#) page for the full flag list; this page covers only the right-click Structure-tree access path.

What it needs

Optional, per the feature map's Index column.

Example

Illustrative: with the Structure form open, right-click the node for `TMyClass.DoWork` and choose "Find Usages" to list every reference to `DoWork` across the indexed project.

Fix all in project

Applies autofixes for every fixable finding across the whole project, starting from the Structure form. Reach for it when you want a project-wide sweep of autofixes in one action instead of fixing unit by unit.

Reaching it in the IDE

This is a RIGHT-CLICK action, not a main-menu item. Menu path: "Structure form (right-click a tree node)".

The Structure form's right-click menu is the only place autofix ("Fix it", "Fix all in unit", "Fix all in project") is reachable in the IDE -- there is no main drag-lint menu item for it.

To reach it: open the Structure form first (drag-lint > Show Structure), then right-click a tree node and choose "Fix all in project".

Running it from the CLI

The menu item runs `lint-all` with the autofix flags across the whole indexed project:

```
drag-lint lint-all --db <project.sqlite> --fix --apply
```

Without `--apply` the command is a DRY RUN. Dry-run first -- this one can rewrite many files at once.

Known limitation, from the plugin source: after a project-wide fix only the CURRENT file's editor buffer is reloaded. Other edited files stay stale in their open editor buffers until you re-open them.

What it needs

The `--db` flag is optional -- drag-lint auto-resolves the project's index when it is omitted. But `lint-all` does read an index, so one must exist. The project's DB lives at `<project folder>\_D-RAG\<project file>.sqlite`; run `drag-lint resolve-dbs --project <X.dproj>` if you are unsure which one.

Example

Dry run:

```
drag-lint lint-all --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --fix
```

Apply for real:

```
drag-lint lint-all --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --fix --apply
```

Fix all in unit

Applies autofixes for every fixable finding in the current unit, starting from the Structure form. Reach for it when a unit has several fixable findings and you want them all resolved in one action.

Reaching it in the IDE

This is a RIGHT-CLICK action, not a main-menu item. Menu path: "Structure form (right-click a tree node)".

The Structure form's right-click menu is the only place autofix ("Fix it", "Fix all in unit", "Fix all in project") is reachable in the IDE -- there is no main drag-lint menu item for it.

To reach it: open the Structure form first (drag-lint > Show Structure), then right-click a tree node and choose "Fix all in unit".

Running it from the CLI

The menu item runs `lint` with the autofix flags, unscoped -- no `--fix-line/--fix-rule`, so every fixable finding in the file is applied:

```
drag-lint lint --file <path.pas> --fix --apply
```

Without `--apply` the command is a DRY RUN: it reports what it would fix and writes nothing. Always dry-run first.

Only rules marked fixable are applied; 22 of the 173 rules are fixable (`drag-lint rules --json` reports a `fixable` field per rule).

What it needs

No index required -- `lint --file` works on a single file.

Example

Dry run:

```
drag-lint lint --file C:\Projects\MyApp\Unit1.pas --fix
```

Typical output when nothing is fixable:

autofix: no fixable findings (of 13 finding(s))

Apply for real:

```
drag-lint lint --file C:\Projects\MyApp\Unit1.pas --fix --apply
```


Fix it

Applies the autofix for the single lint finding represented by a Structure-tree node. Reach for it when you are looking at one finding and want it fixed without touching anything else in the unit.

Reaching it in the IDE

This is a RIGHT-CLICK action, not a main-menu item. Menu path: "Structure form (right-click a tree node)".

The Structure form's right-click menu is the only place autofix ("Fix it", "Fix all in unit", "Fix all in project") is reachable in the IDE -- there is no main drag-lint menu item for it.

To reach it: open the Structure form first (drag-lint > Show Structure), then right-click the tree node for a finding and choose "Fix it".

Running it from the CLI

The menu item runs `lint` with the autofix flags:

```
drag-lint lint --file <path.pas> --fix --fix-line <L> --fix-rule <rule-id> --apply
```

`--fix-line` and `--fix-rule` narrow the fix to the ONE finding you right-clicked. Without `--apply` the command is a DRY RUN: it reports what it would fix and writes nothing.

Only rules marked fixable can be auto-fixed. `drag-lint rules --json` reports a `fixable` field per rule; 22 of the 173 rules are fixable.

What it needs

No index required -- `lint --file` works on a single file. (An index is still needed for the Structure form itself to populate.)

Example

Dry run first, to see what would change:

```
drag-lint lint --file C:\Projects\MyApp\Unit1.pas --fix
```

Typical output when nothing is fixable:

autofix: no fixable findings (of 13 finding(s))

Then apply one specific finding:

```
drag-lint lint --file C:\Projects\MyApp\Unit1.pas --fix --fix-line 42 --fix-rule bare-  
except --apply
```

Format Whole Project with YADF

Runs YADF formatting across a project's files, with indexing involved as part of the same action. Reach for it to bring a whole project's source to house style in one action instead of file by file.

Running it from the CLI

No single combined command is documented for this action. The feature map lists its CliVerb as "format + index" and its Mechanism as "lsp+shells", meaning it is built from the `format` verb (run per file) and the `index` verb, not one dedicated whole-project-format command:

```
drag-lint format <file> [--yadf-path PATH]
drag-lint index <path> [--db <file.sqlite>] [--watch [--interval N]] [--library-db
<lib.sqlite> ...]
```

No documented flag runs both across a whole project in a single CLI call; that orchestration is the IDE menu action itself.

Reaching it in the IDE

drag-lint > Format Whole Project with YADF...

What it needs

The feature map marks this row's Index column "no/optional": the formatting part needs no index. Where indexing is involved, the `--db` flag on `index` is optional -- drag-lint auto-resolves the index when omitted -- but an index must still exist for that step to do anything.

Formatting does NOT invalidate your review markers

This is the question people ask before letting a formatter loose on a reviewed codebase, so it was tested rather than assumed.

// drag-lint:ignore comments and // dl:ok <rule>@<hash> reviewed markers **survive YADF formatting**. A `dl:ok` marker carries a 4-char hash of the line's code so that a review cannot outlive the code it reviewed -- but that hash is computed over a NORMALISED line: comments and whitespace stripped, identifiers lowercased, string-literal content preserved verbatim. Reindenting and re-spacing therefore cannot change it.

Measured on `YADFOT.Wizard.pas` (10 markers) before and after a real `drag-lint format` run: all 10 markers survived, and the only change to those lines was whitespace --

```
BEFORE: try UnregisterYADFOptions;  except end;  // dl:ok empty-except@1500, try-except-  
swallowed@1500  
AFTER : try UnregisterYADFOptions; except end;  // dl:ok empty-except@1500, try-except-  
swallowed@1500
```

Note what did NOT happen: the one-line `try ... except end;` statements were not rewrapped. That is the case that would matter, because splitting or joining a line changes the normalised content and WOULD re-hash the marker. So the guarantee is precisely "reindentation and re-spacing are safe" -- not "any transformation is safe".

Example

Illustrative only, showing the per-file and index verbs this action is built from:

```
drag-lint format C:\Projects\MyApp\Unit1.pas  
drag-lint index C:\Projects\MyApp --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

Format with YADF

Formats a single file's source using YADF (Yet Another Delphi Formatter). Reach for it to reformat the current file to house style without leaving the IDE.

Running it from the CLI

```
drag-lint format <file> [--yadf-path PATH]
```

`<file>` is the file to format; `--yadf-path` overrides the YADF executable location if it is not on the default path.

Reaching it in the IDE

drag-lint > Format with YADF

What it needs

No index needed. This action formats the file directly; the feature map marks this row's Index column "no". It needs a file to format -- the current editor buffer's file in the IDE, or a path given on the command line.

Example

Illustrative only:

```
drag-lint format C:\Projects\MyApp\Unit1.pas
```

This would reformat `Unit1.pas` in place using YADF.

Full Compile Sweep

Recompiles a project's stale units and refreshes its stored compiler findings, forcing a full build rather than the default stale-count threshold. Reach for it when you want a complete, current set of compiler diagnostics for a project, not just whatever happened to go stale.

Running it from the CLI

```
drag-lint refresh-findings --project <X.dproj> --db <db> [--full] [--json]
```

--project is the .dproj to check; --db is the project's index database. Without --full, the command only forces a full build once 2 or more units are stale (otherwise it recompiles just the stale ones). This action runs the verb with --full set, forcing a full recompile every time. --json selects JSON output.

Reaching it in the IDE

drag-lint > Full Compile Sweep

What it needs

An index IS required. The project's DB lives at <project folder>_D-RAG\<project file>.sqlite; run drag-lint resolve-dbs --project <X.dproj> if unsure.

Example

Illustrative only:

```
drag-lint refresh-findings --project C:\Projects\MyApp\MyApp.dproj --db  
C:\Projects\MyApp\_D-RAG\MyApp.sqlite --full
```

This would force a full recompile of MyApp.dproj and refresh its stored compiler findings.

Generate Test Helper CSV

Produces a CSV listing of a project's forms, one row per form, for use as a test-helper navigation map. Reach for it when building or updating automated UI test scaffolding that needs to enumerate a project's forms.

Running it from the CLI

```
drag-lint forms-csv --project <X.dproj> --db <file.sqlite> [--out <f.csv>] [--root <TfrmMAIN>]
```

--project is the .dproj to enumerate forms for; --db is the project's index database; --out sets the output CSV path; --root sets the root form to start from.

Reaching it in the IDE

drag-lint > Generate Test Helper CSV...

What it needs

An index IS required. The project's DB lives at <project folder>_D-RAG\<project file>.sqlite; run drag-lint resolve-dbs --project <X.dproj> if unsure.

Example

Illustrative only:

```
drag-lint forms-csv --project C:\Projects\MyApp\MyApp.dproj --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --out C:\Projects\MyApp\forms.csv --root TfrmMain
```

This would write one CSV row per form in MyApp.dproj, starting from TfrmMain, to forms.csv.

Go to Declaration

Jumps the editor caret to the declaration of the symbol represented by a Structure-tree node. Reach for it when you are browsing the Structure outline and want to land on where a symbol is declared, without first clicking into the editor.

Reaching it in the IDE

This is a RIGHT-CLICK action, not a main-menu item. Menu path: "Structure form (right-click a tree node)".

To reach it: open the Structure form first (drag-lint > Show Structure), then right-click a node in the tree and choose "Go to Declaration".

Running it from the CLI

No CLI equivalent - this is an editor navigation action.

What it needs

Not needed. The feature map marks this row's Index column "n/a" and its Mechanism "in-process" -- it does not shell out to the CLI or query an index database.

Example

Illustrative: with the Structure form open and showing the current unit, right-click the node for `TMyForm.ButtonClick` and choose "Go to Declaration". The editor jumps to the line where `TMyForm.ButtonClick` is declared.

Go to Definition

Jumps the editor caret to the declaration of the symbol under the cursor. Reach for it to navigate straight to where a symbol is defined.

Reaching it in the IDE

drag-lint > Go to Definition

No keyboard shortcut is documented for this menu item.

Running it from the CLI

There is no standalone CLI verb for this (feature map `CliVerb: "(none - in-process)"`). It is served in-process by the IDE plugin. The bundled language server exposes the same capability over stdio to other editors -- the [Features](#) page lists go-to-definition among the methods `drag-lint lsp` implements -- but that is a stdio protocol server, not a one-shot CLI command.

What it needs

The feature map marks this row's Index column "n/a" and its Mechanism "in-process" -- it does not shell out to the CLI or query an index database through a verb. It needs an open editor with the caret on a symbol.

Example

Illustrative: with the caret on a reference to `TCustomerOrder.Total` in the editor, choose "Go to Definition" to jump to where `Total` is declared.

Go to Implementation

Jumps the editor caret to the implementation of the symbol represented by a Structure-tree node. Reach for it when the tree node you are on is a declaration (for example an interface method or a forward-declared routine) and you want the actual implementation body instead.

Reaching it in the IDE

This is a RIGHT-CLICK action, not a main-menu item. Menu path: "Structure form (right-click a tree node)".

To reach it: open the Structure form first (drag-lint > Show Structure), then right-click a node in the tree and choose "Go to Implementation".

Running it from the CLI

No CLI equivalent - this is an editor navigation action.

What it needs

Not needed. The feature map marks this row's Index column "n/a" and its Mechanism "in-process" -- it does not shell out to the CLI or query an index database.

Example

Illustrative: with the Structure form open, right-click the node for `TMyClass.DoWork` and choose "Go to Implementation". The editor jumps to the procedure `TMyClass.DoWork;` body in the implementation section.

Hover at Cursor

Shows a hover card for the symbol under the editor caret: signature, documentation, and callers. Reach for it to check what a symbol is and how it is used without leaving the line you are on.

Reaching it in the IDE

drag-lint > Hover at Cursor

Keyboard shortcut: Ctrl+Alt+H.

Running it from the CLI

In the IDE this action is served in-process (the feature map's Mechanism column marks it "in-process") -- no exe is spawned per hover. The same capability is also exposed as a standalone CLI verb, and over stdio by the bundled language server (the [Features](#) page lists hover among the methods `drag-lint lsp` implements):

```
drag-lint hover --qname <Foo.Bar> [--db <file.sqlite>] [--format plain|md|json]
```

`--qname` is the fully qualified symbol; `--format` selects plain text, markdown, or JSON.

What it needs

The `--db` flag is optional -- drag-lint auto-resolves the index when omitted. An index must still exist. In the IDE, it also needs an open editor with the caret on a symbol.

Example

Illustrative only:

```
drag-lint hover --qname CustomerOrder.TCustomerOrder.Total --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --format md
```

This would print the hover card for the `Total` member of `TCustomerOrder` as markdown.

Impact / Blast Radius (symbol)

Reports what would be affected if a chosen symbol changes -- the set of code to retest. Reach for it before changing a widely-used method or type.

Running it from the CLI

```
drag-lint impact --qname <Foo.Bar> [--db <file.sqlite>] [--depth N] [--format text|json]
```

--qname is the fully qualified symbol; --depth bounds how far impact is traced; --format selects text or json.

Reaching it in the IDE

drag-lint > Uses & Dependencies > Impact / Blast Radius (symbol)...

What it needs

Optional. An index is not marked required, but the impact trace is read from whatever has been indexed, so a stale or missing index gives an incomplete answer.

Example

Illustrative only:

```
drag-lint impact --qname CustomerOrder.TCustomerOrder.Total --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --depth 3
```

This would report what is affected, up to 3 levels out, if the Total member of TCustomerOrder changes.

Import Build Log

Parses an external dcc or msbuild build log and folds its errors and warnings into drag-lint's stored findings. Reach for it to bring diagnostics from a build that ran outside drag-lint (for example a CI build) into the index.

Running it from the CLI

```
drag-lint import-log <logfile> --db <file.sqlite>
```

<logfile> is the dcc/msbuild log to parse; --db is the index database to write the findings into.

Reaching it in the IDE

drag-lint > Import Build Log...

What it needs

An index IS required. The project's DB lives at <project folder>_D-RAG\<project file>.sqlite; run `drag-lint resolve-dbs --project <X.dproj>` if unsure.

Example

Illustrative only:

```
drag-lint import-log C:\Projects\MyApp\build.log --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would parse `build.log` and add its errors and warnings to `MyApp.sqlite`'s stored findings.

Library Drift Check

Flags registered library roots that have source on disk but nothing in the index. Reach for it to catch a library index that has silently gone stale or was never built.

Running it from the CLI

```
drag-lint library-drift [--platform <p>] [--config <path>] [--json]
```

Exits 2 if drift is found. `--platform` scopes the check to `win32` or `win64`; `--config` points at an alternate manifest.

Reaching it in the IDE

drag-lint > Index & Maintenance > Library Drift Check...

What it needs

No index required -- this command has no `--db` flag; it checks the registered library manifest and what is on disk against what is already indexed, rather than querying a specific project's index.

Example

Illustrative only:

```
drag-lint library-drift --platform win64 --json
```

This would report, as JSON, any Win64 library roots that have source on disk but nothing indexed, exiting 2 if any are found.

Lint Buffer (Unsaved)

Lints the current editor buffer's in-memory content, including unsaved edits, without writing it to disk first. Reach for it to see lint findings for changes you have not saved yet.

Running it from the CLI

The feature map lists this action's CliVerb as `lint`, but its Mechanism is "in-process" -- the IDE lints the in-memory buffer directly rather than shelling out to the CLI. The nearest CLI equivalent operates on a saved file on disk:

```
drag-lint lint <path> [--rule <id>] [--disable id1,id2] [--rules-dir <dir>] [--json]
```

Reaching it in the IDE

```
drag-lint > Lint Buffer (Unsaved)
```

What it needs

No index needed. This action lints the buffer's content directly; the feature map marks this row's Index column "no".

Example

Illustrative only, showing the CLI verb this action's findings correspond to once a file is saved:

```
drag-lint lint C:\Projects\MyApp\Unit1.pas --json
```

The IDE action produces the equivalent findings for the unsaved buffer, in-process, without this shell-out.

Open Plugin Log

Opens the drag-lint plugin's own log file. Reach for it when diagnosing the plugin's own behavior -- what it did, what it shelled out to, and any errors it hit.

Running it from the CLI

No CLI equivalent. The feature map lists this action's CliVerb as "(none - in-process)" and its Mechanism as "in-process".

Reaching it in the IDE

drag-lint > Open Plugin Log

What it needs

Not documented as needing an index -- the feature map marks this row's Index column "n/a". It needs the plugin to have written a log file to open.

Example

Illustrative only: choosing drag-lint > Open Plugin Log opens the plugin's log file in an editor or viewer, showing recent plugin activity.

Quick-Fix: Add Unit for Inline Hint (H2443) at Cursor

The same idea as the undeclared-identifier quick-fix, but for compiler hint H2443 -- an inline routine whose declaring unit is not in scope. Reach for it when the compiler flags an inline expansion that needs another unit.

Running it from the CLI

No CLI equivalent - this action is in-process only.

Reaching it in the IDE

drag-lint > Uses & Dependencies > Quick-Fix: Add Unit for Inline Hint (H2443) at Cursor

What it needs

n/a. This action runs in-process against the open editor buffer; the feature map does not mark it as needing a separate index step.

Example

Illustrative only: with an H2443 hint reported at the caret -- say, on a call that resolves to an inline method of `TCustomerOrder` -- invoke the quick-fix to add the unit that declares it to `uses`.

Quick-Fix: Add Unit for Undeclared at Cursor (Ctrl+Alt+U)

Finds which unit declares the identifier under the caret and adds that unit to the current unit's `uses` clause. Reach for it right where the compiler would otherwise report an undeclared identifier.

Running it from the CLI

No CLI equivalent - this action is in-process only.

Reaching it in the IDE

drag-lint > Uses & Dependencies > Quick-Fix: Add Unit for Undeclared at Cursor (Ctrl+Alt+U)

What it needs

n/a. This action runs in-process against the open editor buffer; the feature map does not mark it as needing a separate index step.

Example

Illustrative only: place the caret on an undeclared identifier -- for example a reference to `TCustomerOrder` in a unit that has not yet added the unit that declares it -- and invoke the quick-fix (or press Ctrl+Alt+U) to add the declaring unit to `uses`.

Quick-Fix: Convert Public Field to Property at Cursor

Rewrites a public field under the caret as a property. Reach for it when tightening up a class's public surface.

Running it from the CLI

No CLI equivalent - this action is in-process only.

Reaching it in the IDE

drag-lint > Uses & Dependencies > Quick-Fix: Convert Public Field to Property at Cursor

What it needs

n/a. This action runs in-process against the open editor buffer; the feature map does not mark it as needing a separate index step.

Example

Illustrative only: place the caret on a public field -- for example `FCustomer: TCustomerOrder;` exposed directly on `TfrmMain` -- and invoke the quick-fix to convert it into a property.

Rebuild Index for This Project

Walks a project's `.dproj` compile closure and (re)builds its index. Reach for it after adding or moving units, or when hover, Find Usages, or lint results feel stale.

Running it from the CLI

```
drag-lint index --project <file.dproj> [--db <file.sqlite>] [--dry-run] [--watch [--interval N]]
```

`--dry-run` previews without writing; `--watch [--interval N]` keeps re-indexing on an interval. Mode is chosen per run: `--recompile` (default) updates the index in place; `--rebuild` first empties it of source, then walks (implies `--force-reparse`). `--force-reparse` (alias `--no-skip`) re-parses every walked file even when unchanged. `--prune` forces the delete-sweep for a single-file walk; `--no-prune` computes and reports what would be removed without deleting anything.

Reaching it in the IDE

drag-lint > Index & Maintenance > Rebuild Index for This Project

What it needs

The `--db` flag is optional -- drag-lint auto-resolves the project's index when omitted. An index must still exist.

Example

Illustrative only:

```
drag-lint index --project C:\Projects\MyApp\MyApp.dproj --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would re-index MyApp's compile closure in place (default recompile mode).

Reconcile Project Members (.dpr/.dproj)

Compares a project file's declared member list against what is on disk and in the compile closure, and can also re-scan and recompile every member to heal the index and findings. Reach for it after moving or adding units by hand.

Running it from the CLI

```
drag-lint reconcile-project <App.dpr|.dproj> [--apply] [--db <db>] [--full] [--json] [--config <path>]
```

`--apply` writes the sync; without it the command reports only. `--db` also heals the index and findings for every project member (re-scan + recompile) without editing the `.dpr`; `--full` forces the recompile even when nothing looks incoherent.

Reaching it in the IDE

drag-lint > Uses & Dependencies > Reconcile Project Members (.dpr/.dproj)...

What it needs

Optional. An index is not required to run this command, but pass `--db` to also heal the index and findings for the project's members.

Example

Illustrative only:

```
drag-lint reconcile-project C:\Projects\MyApp\MyApp.dproj --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would compare `MyApp.dproj`'s member list against disk and the compile closure, and re-scan/recompile its members -- one of which might be the unit that owns `TfrmMain`.

Recover Buffer-Compile Files

Restores any file left holding unsaved-buffer content after a Compile Buffer (unsaved) run was interrupted (for example by a crash). Reach for it, or let the IDE run it automatically, so no unit is left holding overlaid content instead of its saved original.

Running it from the CLI

The feature map lists this action's CliVerb as "(none - in-process)", but this IDE action actually runs the `ghost-recover` verb. It prints no usage banner: running it bare exits 0 and prints `ghost-recover: nothing pending`. when there is nothing to recover. The plugin invokes it as:

```
drag-lint ghost-recover "<project>"
```

Reaching it in the IDE

drag-lint > Recover Buffer-Compile Files

What it needs

The feature map marks this row's Index column "n/a". It needs the project path so it can look for pending recovery state for that project.

Example

Illustrative only:

```
drag-lint ghost-recover "C:\Projects\MyApp\MyApp.dproj"
```

This would check for any file left overlaid by an interrupted Compile Buffer (unsaved) run and restore it to its saved original.

Rename Symbol...

Renames a symbol across the whole indexed project. Reach for it instead of a text find/replace when a symbol appears in many units, so every reference is updated together.

Reaching it in the IDE

drag-lint > Rename Symbol...

Keyboard shortcut: Ctrl+Alt+R.

Running it from the CLI

The feature map's Mechanism column marks this row "delegates" -- the menu item runs the CLI `rename` verb rather than an in-process edit:

```
drag-lint rename --kind symbol --name <QName> --to <New> [--json|--apply|--no-backup] --db <db>
```

Two related forms also exist: `--kind param --file <F> --line <L> --col <C> --to <New> [...]` for a routine-local parameter/var rename, and `--qname <Foo.TBar.Baz> --to <NewName> [--db PATH] [--dry-run] [--no-backup]`. Without `--apply` (or with `--dry-run`), the run is a preview only.

What it needs

An index IS required. The project's DB lives at `<project folder>\_D-RAG\<project file>.sqlite`; run `drag-lint resolve-dbs --project <X.dproj>` if unsure.

Example

Illustrative only:

```
drag-lint rename --kind symbol --name CustomerOrder.TCustomerOrder.Total --to GrandTotal --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --apply
```

This would rename `Total` to `GrandTotal` everywhere it is referenced in the index.

Reverse Call Tree (clickable, Messages window)

The same reverse call tree, rendered into the IDE's Messages window so each line navigates to the caller. Reach for it when you want to click through the callers instead of reading a flat report.

Running it from the CLI

```
drag-lint reverse-calltree --qname <X> [--direction callers|callees] [--depth N] [--format text|json|dot|mermaid] [--json] --db PATH [--db ...]
```

Same command as the plain Reverse Call Tree; the IDE renders the result into the clickable Messages window instead of a text report.

Reaching it in the IDE

drag-lint > Uses & Dependencies > Reverse Call Tree (clickable, Messages window)...

What it needs

An index IS required -- `reverse-calltree` reads the project's index database. The project's DB lives at `<project folder>\_D-RAG\<project file>.sqlite`; run `drag-lint resolve-dbs --project <X.dproj>` if you are unsure which one.

Example

Illustrative only: invoking this on `CustomerOrder.TCustomerOrder.Total` would list its callers in the Messages window, each line clickable to jump to that call site.

Reverse Call Tree (who calls this, N-deep)

Builds the tree of callers of a chosen symbol, transitively, to a chosen depth. Reach for it to see how far a change to one routine could ripple backward through its callers.

Running it from the CLI

```
drag-lint reverse-calltree --qname <X> [--direction callers|callees] [--depth N] [--format text|json|dot|mermaid] [--json] --db PATH [--db ...]
```

Default direction is `callers` (who calls X); `--direction callees` instead shows what X calls. `--depth` bounds the tree; `--format` selects text, json, dot, or mermaid output. The command is cycle-guarded.

Reaching it in the IDE

drag-lint > Uses & Dependencies > Reverse Call Tree (who calls this, N-deep)...

What it needs

An index IS required -- `reverse-calltree` reads the project's index database. The project's DB lives at `<project folder>\_D-RAG\<project file>.sqlite`; run `drag-lint resolve-dbs --project <X.dproj>` if you are unsure which one.

Example

Illustrative only:

```
drag-lint reverse-calltree --qname CustomerOrder.TCustomerOrder.Total --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --depth 2
```

This would show, up to 2 levels deep, who calls the `Total` member of `TCustomerOrder`.

Run AST Checks

Runs drag-lint's AST-level checks against a file. Reach for it to validate a file's parse tree and catch AST-detectable issues directly.

Running it from the CLI

```
drag-lint check-ast <file> [--db PATH] [--format text|json]
```

`<file>` is the file to check; `--db` is an index database; `--format` selects `text` or `json` output.

Reaching it in the IDE

drag-lint > Run AST Checks

What it needs

The `--db` flag is optional -- drag-lint auto-resolves the index when omitted. An index must still exist.

Example

Illustrative only:

```
drag-lint check-ast C:\Projects\MyApp\Unit1.pas --format json
```

This would run AST checks on `Unit1.pas` and print the results as JSON.

Run Diagnostics (didSave)

Runs drag-lint's diagnostics pass on a file automatically when it is saved, mirroring the LSP `textDocument/didSave` notification. Reach for it indirectly -- it is not something you invoke by hand; it fires on save.

Running it from the CLI

No CLI equivalent. The feature map lists this action's `ClVerb` as "(none - in-process)" and its `Mechanism` as "in-process" -- it runs inside the plugin without shelling out to the drag-lint executable.

Reaching it in the IDE

drag-lint > Run Diagnostics (didSave)

What it needs

Not documented as needing an index -- the feature map marks this row's `Index` column "n/a". Since it fires on save, it needs an open editor with a saved file.

Example

Illustrative only: saving a unit in the editor triggers this action, which runs diagnostics on the just-saved file and updates its findings.

Run Lint All (Full Report)

Runs the full rule catalog (173 rules, 149 on by default across 16 categories, 22 auto-fixable) against a project's indexed code and reports every surviving finding. Reach for it as the overall project health check.

Running it from the CLI

```
drag-lint lint-all [--db <file.sqlite>] [--project <.dproj>] [--disable id,...] [--output <report.txt>] [--json] [--quiet] [--lint-third-party]
```

--project <.dproj|.dpr> reports only on the units that project compiles (its compile closure + their .dfm siblings) -- use it when one folder holds several projects, or when the DB spans more than the project you are reviewing. --quiet suppresses per-file progress lines written to stderr. --lint-third-party restores the old unscoped whole-index count instead of only the project's own roots. --disable id,... turns off specific rule ids; --output writes the report to a file; --json emits JSON.

The shared Output/CI flags also apply: --format sarif, --fail-on <level>, --config <file>, --enable id1,id2, --profile <name>, --baseline <file>, --write-baseline <file>.

Reaching it in the IDE

drag-lint > Code Quality > Run Lint All (Full Report)...

What it needs

The --db flag is optional -- drag-lint auto-resolves the project's index when omitted. An index must still exist.

Example

Illustrative only:

```
drag-lint lint-all --project C:\Projects\MyApp\MyApp.dproj --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --output report.txt
```

This would report every surviving finding across MyApp's own compile closure and write it to report.txt.

Scan TODOs / FIXMEs

Scans source for TODO/FIXME/HACK/XXX/REVIEW/NOTE markers. Reach for it to triage outstanding work markers left in code.

Running it from the CLI

```
drag-lint todos [<path>] [--json]
```

<path> is optional per the usage banner; --json emits JSON instead of text. No further defaults for an omitted <path> are documented.

Reaching it in the IDE

drag-lint > Code Quality > Scan TODOs / FIXMEs...

What it needs

No index required -- this command has no --db flag; it scans source files directly for TODO/FIXME/HACK/XXX/REVIEW/NOTE markers.

Example

Illustrative only:

```
drag-lint todos C:\Projects\MyApp\src --json
```

This would list TODO/FIXME/HACK/XXX/REVIEW/NOTE markers found under C:\Projects\MyApp\src, as JSON.

Show Completion

Shows an index-backed completion list at the editor caret. Reach for it to see what identifiers are valid to type next.

Reaching it in the IDE

drag-lint > Show Completion

Keyboard shortcut: Ctrl+Alt+C.

Running it from the CLI

There is no standalone CLI verb for this (feature map `CliVerb: "(none - in-process)"`). It is served in-process by the IDE plugin. The bundled language server exposes completion over stdio to other editors -- the [Features](#) page lists completion among the methods `drag-lint lsp` implements -- but that is a stdio protocol server, not a one-shot CLI command.

What it needs

The feature map marks this row's Index column "n/a" and its Mechanism "in-process". It needs an open editor buffer with the caret positioned where completion is requested.

Example

Illustrative: type a partial identifier in the editor and press Ctrl+Alt+C to show the completion list for what follows.

Show in Call Graph

Opens the call graph rooted at the symbol represented by a Structure-tree node. Reach for it when you want to see who calls (or is called by) a symbol you are already looking at in the Structure outline.

Reaching it in the IDE

This is a RIGHT-CLICK action, not a main-menu item. Menu path: "Structure form (right-click a tree node)".

To reach it: open the Structure form first (drag-lint > Show Structure), then right-click a node in the tree and choose "Show in Call Graph".

Running it from the CLI

The feature map lists the underlying verb as `reverse-calltree`. From the usage banner:

```
drag-lint reverse-calltree --qname <X> [--direction callers|callees] [--depth N] [--format text|json|dot|mermaid] [--json] --db PATH [--db ...]
```

callers = who calls X (default), callees = what X calls; cycle-guarded.

What it needs

An index IS required -- `reverse-calltree` reads the project's index database. The project's DB lives at `<project folder>\_D-RAG\<project file>.sqlite`; run `drag-lint resolve-dbs --project <X.dproj>` if you are unsure which one.

Example

Illustrative:

```
drag-lint reverse-calltree --qname Unit1.TMyClass.DoWork --direction callers --depth 2 --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --format text
```

This would print the callers of `DoWork`, two levels deep.

Show Resolved DBs (debug)

Prints which database(s) drag-lint would actually use for a given project, file, or platform, without running a real query. Reach for it when a query seems to be answering from the wrong index.

Running it from the CLI

```
drag-lint resolve-dbs [--platform win32|win64] [--config <path>] [--json]
drag-lint resolve-dbs --project <file.dproj> [--config <path>] [--json]
drag-lint resolve-dbs --in <file.pas> [--project <file.dproj>] [--config <path>] [--json]
```

The first form prints the consumer DB list query/lsp/serve would use. `--project` prints the one DB that owns that project -- the write target; it exits 2 and names the sections if none or several claim it, rather than guessing. `--in <file.pas>` prints the read list the IDE would open for that file (the active project's DB first, then the folder-matched DB); omit `--project` to model "no project active".

Reaching it in the IDE

drag-lint > Index & Maintenance > Show Resolved DBs (debug)...

What it needs

No index required -- this command has no `--db` flag; it only resolves and prints which database(s) drag-lint would use, from config and the manifest, without querying index content.

Example

Illustrative only:

```
drag-lint resolve-dbs --project C:\Projects\MyApp\MyApp.dproj --json
```

This would print, as JSON, the one database that owns `MyApp.dproj`.

Show Signature Help

Shows parameter help for the call being typed at the editor caret. Reach for it to check a routine's parameter list and order while writing a call.

Reaching it in the IDE

drag-lint > Show Signature Help

Keyboard shortcut: Ctrl+Alt+S.

Running it from the CLI

There is no standalone CLI verb for this (feature map `CliVerb: "(none - in-process)"`). It is served in-process by the IDE plugin. The bundled language server exposes signature help over stdio to other editors -- the [Features](#) page lists signature help among the methods `drag-lint lsp` implements -- but that is a stdio protocol server, not a one-shot CLI command.

What it needs

The feature map marks this row's Index column "n/a" and its Mechanism "in-process". It needs an open editor with the caret inside a call's argument list.

Example

Illustrative: position the caret inside the parentheses of a call to `TCustomerOrder.Create(...)` and press Ctrl+Alt+S to show that constructor's parameter help.

Show Structure

Shows a structural outline of the current unit. Reach for it to see a unit's types and members as a navigable tree instead of scrolling the source.

Reaching it in the IDE

drag-lint > Show Structure

No keyboard shortcut is documented for this menu item.

Running it from the CLI

There is no standalone CLI verb for this (feature map CliVerb: "(none - in-process)"). It is served in-process by the IDE plugin. This is the Structure form that several other feature-map rows (Go to Declaration, Find Usages (context), Show in Call Graph, and others) reach by right-clicking a tree node.

What it needs

The feature map marks this row's Index column "n/a" and its Mechanism "in-process". It needs the unit open in the editor.

Example

Illustrative: with a unit open, choose "Show Structure" to open the Structure form showing that unit's types and members as a tree.

Show Wiring (Spring4D DI + DFM events)

Surfaces bindings the compiler does not make obvious: Spring4D container registrations and DFM event hookups for a type. Reach for it when tracing how an interface gets resolved, or which handler a DFM event actually calls.

Running it from the CLI

```
drag-lint wiring --qname <IIntf|TForm> [--db <file.sqlite>] [--format text|json]
```

A separate coverage mode is also documented:

```
drag-lint wiring --coverage [--db <file.sqlite>] [--format text|json]
```

which reports DI registrations not yet resolved to an interface-to-type pairing.

Reaching it in the IDE

drag-lint > Uses & Dependencies > Show Wiring (Spring4D DI + DFM events)...

What it needs

Optional. An index is not marked required, but the wiring data comes from whatever has been indexed.

Example

Illustrative only:

```
drag-lint wiring --qname MyApp.TfrmMain --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --format text
```

This would list Spring4D DI registrations and DFM event edges touching `TfrmMain`.

Symbol Search...

Searches symbols by name across the indexed projects. Reach for it to jump to a type, method, or field when you know (or can guess) its name but not its file.

Reaching it in the IDE

drag-lint > Symbol Search...

Keyboard shortcut: Ctrl+Alt+T.

Running it from the CLI

```
drag-lint query --name <symbol-name> [--db <file.sqlite>] [--json] [--case-sensitive] [--exact]
drag-lint query --qname <qualified> [--db <file.sqlite>] [--json] [--case-sensitive]
```

--name/--qname match case-insensitively by default (--case-sensitive restores byte-exact matching); --name also accepts a qualified name. --exact suppresses the fuzzy fallback, so zero rows means "no such symbol". The same lookup is also exposed over stdio as the language server's "workspace symbols" method (the [Features](#) page lists it under drag-lint lsp).

What it needs

The --db flag is optional -- drag-lint auto-resolves the index when omitted. An index must still exist.

Example

Illustrative only:

```
drag-lint query --name TCustomerOrder --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --json
```

This would return every symbol named TCustomerOrder (exact or fuzzy) as JSON.

Symbol Slice...

Shows the slice of code relevant to one symbol: its declaration, body, and immediate context. Reach for it to see just what matters for a symbol instead of the whole file.

Reaching it in the IDE

drag-lint > Inspect Symbol > Symbol Slice...

No keyboard shortcut is documented for this menu item.

Running it from the CLI

```
drag-lint slice --qname <Foo.TBar> [--db <file.sqlite>] [--format text|json]
```

--qname is the fully qualified symbol; --format selects text or JSON.

What it needs

The --db flag is optional -- drag-lint auto-resolves the index when omitted. An index must still exist.

Example

Illustrative only:

```
drag-lint slice --qname CustomerOrder.TCustomerOrder.Total --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would print the declaration, body, and immediate context of the `Total` member.

Top Symbols (fan-in)

Ranks indexed symbols by fan-in -- how many places reference them. Reach for it to see the most depended-upon symbols before refactoring one of them.

Running it from the CLI

```
drag-lint top --db <file.sqlite> [--by fanin] [--limit N] [--json]
```

--by fanin selects the ranking metric (the only value shown in the usage banner); --limit caps how many rows are returned; --json emits JSON.

Reaching it in the IDE

drag-lint > Code Quality > Top Symbols (fan-in)...

What it needs

An index IS required. The project's DB lives at <project folder>_D-RAG\<project file>.sqlite; run drag-lint resolve-dbs --project <X.dproj> if unsure.

Example

Illustrative only:

```
drag-lint top --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --by fanin --limit 20
```

This would list MyApp's 20 highest fan-in symbols.

Type at Cursor

Resolves the static type of the expression under the editor caret. Reach for it when you are not sure what type an expression evaluates to.

Reaching it in the IDE

drag-lint > Inspect Symbol > Type at Cursor

No keyboard shortcut is documented for this menu item.

Running it from the CLI

```
drag-lint typeat <file>:<line>:<col> [--db <file.sqlite>] [--format text|json]
```

The position is given as <file>:<line>:<col>; --format selects text or JSON.

What it needs

The --db flag is optional -- drag-lint auto-resolves the index when omitted. An index must still exist.

Example

Illustrative only:

```
drag-lint typeat C:\Projects\MyApp\Orders.pas:42:15 --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would report the static type of the expression at line 42, column 15 of `Orders.pas`.

Unit Test Stub (symbol)

Uses `generate-test` to produce a unit-test skeleton for one qualified symbol. Reach for it to scaffold a test file for a routine that has none.

Running it from the CLI

```
drag-lint generate-test --qname <Foo.TBar.Baz> [--framework dunitx|dunit] [--db PATH]
```

`--qname` is the fully qualified symbol; `--framework` selects `dunitx` or `dunit`.

Reaching it in the IDE

drag-lint > Generate & Export > Unit Test Stub (symbol)...

What it needs

The `--db` flag is optional -- drag-lint auto-resolves the project's index when omitted. An index must still exist.

Example

Illustrative only:

```
drag-lint generate-test --qname MyApp.TCustomerOrder.Total --framework dunitx --db  
C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would produce a DUnitX test stub targeting the `Total` member of `TCustomerOrder`.

Uses Audit -- interface->impl moves + unused (this unit)

For one unit, reports which `uses` entries could move from the `interface` section to `implementation`, and which are unused entirely. Reach for it when cleaning up a single unit's `uses` clauses before a rename or refactor.

Running it from the CLI

```
drag-lint uses-audit <unit.pas> --db <file.sqlite> [--format json|text]
```

Reaching it in the IDE

drag-lint > Uses & Dependencies > Uses Audit -- interface->impl moves + unused (this unit)...

What it needs

Required. You must have indexed the project first -- the audit resolves symbol usage against the index, not by re-parsing the unit alone.

Example

Illustrative only:

```
drag-lint uses-audit C:\Projects\MyApp\CustomerOrder.pas --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --format text
```

This would report, for `CustomerOrder.pas`, which units in its `uses` clause (for example one only referenced by a class such as `TCustomerOrder`) could move to `implementation`, and which are not referenced at all.

Uses Cleanup Preview (compiler-verified, this unit)

The removals a Uses Audit suggests, but verified by actually compiling -- so a unit only needed for an inline routine or a `{IF}` branch is not stripped by mistake. Reach for it before applying an audit's suggestions.

Running it from the CLI

```
drag-lint uses-fix <unit.pas> --project <dproj> --db <file.sqlite> [--platform win32|win64] [--apply] [--remove-unused]
```

Without `--apply` this previews the cleanup; `--apply` writes it. `--remove-unused` extends the cleanup to unused units; `--platform` selects win32 or win64.

Reaching it in the IDE

drag-lint > Uses & Dependencies > Uses Cleanup Preview (compiler-verified, this unit)...

What it needs

Required. You must have indexed the project first.

Example

Illustrative only:

```
drag-lint uses-fix C:\Projects\MyApp\CustomerOrder.pas --project C:\Projects\MyApp\MyApp.dproj --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --platform win32
```

This would preview the compiler-verified uses cleanup for `CustomerOrder.pas` without changing the file; add `--apply` to write it.

Uses Report (CSV)

Exports the project's unit-dependency data as a CSV file. Reach for it when you need the `uses` graph outside the IDE -- in a spreadsheet or another tool.

Running it from the CLI

```
drag-lint uses-report --output <out.csv> [--db ...] [--depth N] [--include-external] [--all-sources] [--name <pattern>]
```

`--output` is where the CSV is written; `--depth` bounds how far the report follows dependencies; `--include-external` adds units outside the project; `--all-sources` and `--name <pattern>` broaden or filter the scope.

Reaching it in the IDE

drag-lint > Uses & Dependencies > Uses Report (CSV)...

What it needs

Optional. An index is not required per the feature map, but the report is built from indexed data, so results depend on what has been indexed.

Example

Illustrative only:

```
drag-lint uses-report --output C:\Projects\MyApp\uses-report.csv --db  
C:\Projects\MyApp\_D-RAG\MyApp.sqlite --depth 2
```

This would write a CSV of unit dependencies up to 2 levels deep, which could include the unit that declares `TCustomerOrder`.

Part III -- The command line

allow

Records a `dl:ok` review of one finding, so it stops being reported without being fixed. Reach for it when a lint finding is a deliberate, reviewed exception rather than something you intend to change.

Running it from the CLI

```
drag-lint allow <file> --fix-line <L> --fix-rule <id> [--apply]
```

`--fix-line` and `--fix-rule` identify the exact finding to allow. Per the usage banner, this is a dry run without `--apply`: "record a `dl:ok` review of ONE finding; dry-run without `--apply`".

Reaching it in the IDE

Reachable in the IDE through the Structure form's right-click menu -- see [Allow this message](#) for the IDE-side walkthrough (drag-lint > Show Structure, then right-click a finding and choose "Allow this message"). There is no main-menu item; the feature map's internal call site is `DragLint.Plugin.StructureForm.pas:1061`.

What it needs

No index required -- `allow` records the review directly against the given file (a `dl:ok` marker), without going through a project index database.

Example

Illustrative:

```
drag-lint allow C:\Projects\MyApp\Unit1.pas --fix-line 42 --fix-rule unused-private-member --apply
```

This would record `Unit1.pas` line 42's `unused-private-member` finding as reviewed, so it no longer appears in future lint runs.

ambiguous-calls

Resolver-coverage diagnostic that reports call sites the engine could not pin to exactly one target -- unresolved or ambiguous calls. Reach for it when you suspect `find-callees/callgraph/call-edge` answers are missing edges because the resolver gave up on some call sites.

Running it from the CLI

```
drag-lint ambiguous-calls [--qname <Foo.Bar>|--file <file>] --db PATH [--json]
```

Scope the scan to one routine with `--qname`, or to one file with `--file`; omit both to scan the whole database. `--json` emits machine-readable output.

This is labeled a "resolver-coverage diagnostic" in the CLI banner -- it is a troubleshooting tool, not a daily driver.

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

An index IS required. The project's DB lives at `<project folder>\_D-RAG\<project file>.sqlite`; run `drag-lint resolve-dbs --project <X.dproj>` if unsure.

Example

Illustrative only:

```
drag-lint ambiguous-calls --qname CustomerOrder.TCustomerOrder.Total --db  
C:\Projects\MyApp\_D-RAG\MyApp.sqlite --json
```

This would list call sites targeting `Total` that the resolver could not resolve to a single certain target.

bench-context

Benchmarks the `context` command by building context bundles for a sample of symbols against a database. Reach for it to measure the cost of generating context bundles, not to inspect any single symbol.

Running it from the CLI

```
drag-lint bench-context [--db <file.sqlite>] [--n N]
```

`--n N` sets how many symbols the benchmark samples; `--db` selects the index. Neither flag's default is documented in the usage line.

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

The `--db` flag is optional -- drag-lint auto-resolves the index when omitted. An index must still exist.

Example

Illustrative only:

```
drag-lint bench-context --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --n 20
```

This would build context bundles for 20 sampled symbols and report benchmark results.

butterfly

Composes a symbol's callers (upward wing) and callees (downward wing) into one combined chart in a single command. Reach for it when you want the full two-directional call graph of a symbol without running two separate queries.

Running it from the CLI

```
drag-lint butterfly --qname <X> [--depth N] [--format dot|mermaid|text|json] [--output F]
--db PATH [--db ...]
```

`--qname <X>` is the symbol to center the chart on. `--depth N` bounds how far each wing walks. `--format` selects dot (the default), mermaid, text, or json. `--output F` writes the chart to a file instead of stdout. `--db PATH` may be repeated.

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature. The IDE's "Call Graph (Butterfly)..." menu item does NOT call this verb: it runs `reverse-calltree` twice instead (once plain for callers, once with `--direction callees`) and composes the two results itself into the dock's Call Graph tab. See [Call Graph \(Butterfly\)](#) for that path.

What it needs

An index IS required. The project's DB lives at `<project folder>\_D-RAG\<project file>.sqlite`; run `drag-lint resolve-dbs --project <X.dproj>` if unsure.

Example

Illustrative only:

```
drag-lint butterfly --qname CustomerOrder.TCustomerOrder.Total --db C:\Projects\MyApp\_D-
RAG\MyApp.sqlite --format dot
```

This would produce one DOT chart showing both who calls `Total` and what `Total` calls.

call-path

Finds the shortest resolved call path from one symbol to another. Reach for it to answer "does A ever reach B, and how" without manually walking a call tree.

Running it from the CLI

```
drag-lint call-path --from <A> --to <B> [--max-depth N] --db PATH [--json]
```

--from <A> and --to are qualified symbol names. --max-depth N bounds the search. Exit code 1 means no path was found. --json emits machine-readable output.

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

An index IS required. The project's DB lives at <project folder>_D-RAG\<project file>.sqlite; run drag-lint resolve-dbs --project <X.dproj> if unsure.

Example

Illustrative only:

```
drag-lint call-path --from TfrmMain.btnSaveClick --to CustomerOrder.TCustomerOrder.Total  
--db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --json
```

This would report the shortest resolved call chain from the button handler down to Total, or exit 1 if no such chain exists.

callgraph

Prints an N-deep resolved call tree for a symbol, in either direction. Reach for it to see who calls a routine or what a routine calls, several levels deep, from the CLI.

Running it from the CLI

```
drag-lint callgraph --qname <X> [--direction callers|callees] [--depth N] --db PATH [--json]
```

--qname <X> is the symbol to root the tree on. --direction selects callers or callees (which one is the default is not documented in the usage line). --depth N bounds how deep the tree walks; the tree is cycle-guarded. --json emits machine-readable output.

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

An index IS required. The project's DB lives at <project folder>_D-RAG\<project file>.sqlite; run drag-lint resolve-dbs --project <X.dproj> if unsure.

Example

Illustrative only:

```
drag-lint callgraph --qname CustomerOrder.TCustomerOrder.Total --direction callers --depth 3 --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --json
```

This would print the 3-deep tree of everything that (transitively) calls Total.

compile-check

Runs a compiler-backed check against a single Delphi project (.dproj) or source file (.pas) and reports the result. Reach for it when you want a compile-style verification of one target without a full project build.

Running it from the CLI

```
drag-lint compile-check <target.dproj|.pas> [--db PATH] [--format json|text]
```

Reaching it in the IDE

CLI+internal: the plugin calls this internally from `DragLint.Plugin.Editor.pas:2037`. There is no menu item for it -- it is invoked in-process by the plugin, not exposed as a user-facing command.

What it needs

The `--db` flag is optional -- drag-lint auto-resolves the index when omitted. An index must still exist.

Example

Illustrative:

```
drag-lint compile-check C:\Projects\MyApp\MyApp.dproj --format json
```

This would compile-check `MyApp.dproj` and print the result as JSON.

context

Returns a compact context bundle for a symbol -- its doc comment, class surface (signatures), the target's own body, and a capped list of callers -- instead of whole source files. Reach for it before editing or explaining a symbol so you read a fraction of the tokens a full-file read would cost.

Running it from the CLI

```
drag-lint context --task "verb qname" [--db <file.sqlite>] [--format md|json|raw] [--max-callers N] [--context N] [--no-docs]
```

`--task "verb qname"` names the symbol and intended action (e.g. `"modify Foo.Bar"`). `--format` selects `md`, `json`, or `raw`. `--max-callers N` caps how many callers are included. `--context N` is present in the usage line; its effect is not documented beyond the flag name. `--no-docs` omits doc comments from the bundle.

Reaching it in the IDE

No menu item calls this directly. The plugin uses it internally from `DragLint.Plugin.Editor.pas:633`; there is no menu item for it. The MCP `get_context_bundle` tool wraps this same engine.

What it needs

The `--db` flag is optional -- drag-lint auto-resolves the index when omitted. An index must still exist.

Example

Illustrative only:

```
drag-lint context --task "modify CustomerOrder.TCustomerOrder.Total" --db  
C:\Projects\MyApp\_D-RAG\MyApp.sqlite --format md
```

This would return `Total`'s doc comment, its class's surface, its own body, and a capped list of callers, as markdown.

convert-apply

Locates `.dfm` component instances that match a `#convert` rule from a conversion-rules file and rewrites all five surfaces: declaration retype, uses-add, `.dfm` re-emit, property/event access-site rewrite, and runtime-creator retype/TODO markers. Reach for it to actually perform a component-type conversion, after validating and scaffolding the rules file.

Running it from the CLI

```
drag-lint convert-apply --unit <F.pas> --rules <file> --db PATH [--db ...] [--only Name1,Name2,...] [--apply] [--no-backup]
```

`--unit <F.pas>` is the unit to convert. `--rules <file>` is the conversion-rules file. `--only Name1,Name2,...` restricts the conversion to named component instances. Without `--apply` this is dry-run only: a preview that writes nothing. `--apply` writes for real, with backups and a `recovery.txt`, unless `--no-backup` is also given. `--db PATH` is required and may be repeated.

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

An index IS required -- `convert-apply` reads the project's index database. The project's DB lives at `<project folder>\_D-RAG\<project file>.sqlite`; run `drag-lint resolve-dbs --project <X.dproj>` if you are unsure which one.

Example

Illustrative only:

```
drag-lint convert-apply --unit C:\Projects\MyApp\frmOrders.pas --rules  
C:\Projects\MyApp\convert-rules\OvcTable-to-cxGrid.rules --db C:\Projects\MyApp\_D-  
RAG\MyApp.sqlite --only tblOrders
```

This would preview (no `--apply`) the conversion of the `tblOrders` component in `frmOrders.pas` across all five surfaces, without writing any changes.

convert-scaffold

Auto-generates a valid conversion-rules file from the real property trees of a from-type and a to-type: a concrete `#link` where exactly one source property matches by leaf-name and type, a `???` marker for ambiguous matches, and a `DROPPED` note for orphaned source properties. Reach for it to get a starting rules file instead of writing one by hand.

Running it from the CLI

```
drag-lint convert-scaffold --from <FromType> --to <ToType> [--out <file>] [--surface dfm|pas] --db PATH [--db ...]
```

`--from/--to` name the source and target types. `--out <file>` is where the generated rules file is written. `--surface` picks the TO-side target bar: default `dfm` (published properties only) or `pas` (published + public, including public fields). A target with `is_writable=false` is never auto-linked on either surface. `--db PATH` is required and may be repeated.

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

An index IS required -- `convert-scaffold` reads the project's index database. The project's DB lives at `<project folder>\_D-RAG\<project file>.sqlite`; run `drag-lint resolve-dbs --project <X.dproj>` if you are unsure which one.

Example

Illustrative only:

```
drag-lint convert-scaffold --from T0vcTable --to TcxGrid --out C:\Projects\MyApp\convert-rules\0vcTable-to-cxGrid.rules --surface dfm --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would write a starting conversion-rules file linking `T0vcTable` properties to `TcxGrid` properties on the DFM surface, flagging ambiguous or orphaned properties for review.

convert-validate

Parses and validates a conversion-rules file (a reFind-superset DSL), checking its `#link/#default` paths against the real property trees. Reach for it before running `convert-scaffold` or `convert-apply`, to confirm a hand-edited or generated rules file is well-formed.

Running it from the CLI

```
drag-lint convert-validate --rules <file> [--from <FromType>] [--to <ToType>] [--print-parsed] [--db PATH ...]
```

`--rules <file>` is the conversion-rules file to validate. `--from/--to` name the source and target types (optional). `--print-parsed` prints the parsed rules. `--db PATH ...` is optional and may be repeated.

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

Optional, per the feature map's Index column.

Example

Illustrative only:

```
drag-lint convert-validate --rules C:\Projects\MyApp\convert-rules\OvcTable-to-cxGrid.rules --from TOvcTable --to TcxGrid --print-parsed
```

This would parse the rules file and report whether its `#link/#default` paths resolve against the real `TOvcTable` and `TcxGrid` property trees.

create-enum-helper

Generates a Byte-family record helper (ToByte/FromByte/ToInteger/ FromInteger/ToString/FromString) for an enum type. Reach for it when you have an enum and want the usual conversion helpers without writing them by hand.

Running it from the CLI

```
drag-lint create-enum-helper --qname <TEnum> [--apply|--json|--no-backup] [--methods <csv>] [--tostring rtti|case] [--db PATH]
```

--methods defaults to all six (tobyte,frombyte,tointeger,frominteger, tostring,fromstring). --tostring defaults to rtti (RTTI GetEnumName) or can be case (explicit case statement). Idempotent: if a helper for the enum already exists anywhere in the index, the action reports exists and makes no edit. The usage line lists --apply as a flag; whether omitting it produces a dry run is not documented in the help text for this verb.

Reaching it in the IDE

Reachable in the IDE only through the Structure form's right-click menu, not the main menu -- see [Create helper class](#) for the IDE-side walkthrough. The feature map's internal call site is `DragLint.Plugin.StructureForm.pas:1226`.

What it needs

The --db flag is optional -- drag-lint auto-resolves the index when omitted. An index must still exist.

Example

Illustrative:

```
drag-lint create-enum-helper --qname Unit1.TColorKind --apply --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would generate a record helper with all six default methods for `TColorKind`.

deps-report

Produces a third-party dependency rollup from an index: which external units/libraries an indexed codebase depends on. Reach for it when auditing external dependencies across a project.

Running it from the CLI

```
drag-lint deps-report --db <file.sqlite> [--db ...] [--depth N] [--edges] [--all-sources]
[--name <pat>] [--format text|json|csv] [--output <file>]
```

--db may be repeated to roll up dependencies across more than one index.

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

An index IS required. The project's DB lives at <project folder>_D-RAG\<project file>.sqlite; run `drag-lint resolve-dbs --project <X.dproj>` if unsure.

Example

Illustrative only:

```
drag-lint deps-report --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --format csv --output
deps.csv
```

This would write MyApp's third-party dependency rollup to `deps.csv`.

diff

Compares two index databases and reports what changed between them -- for example an index snapshot taken before and after a refactor. Reach for it when you want the symbol/reference-level delta between two points in time.

Running it from the CLI

```
drag-lint diff --db <old.sqlite> --db <new.sqlite> [--json]
```

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

An index IS required -- in fact TWO: `diff` takes exactly two `--db` flags, one for the "old" database and one for the "new" one, and both must already exist. Run `drag-lint resolve-dbs --project <X.dproj>` if unsure which DB covers a given project.

Example

Illustrative only:

```
drag-lint diff --db drag-lint-before.sqlite --db drag-lint-after.sqlite --json
```

This would print, as JSON, what changed between the two snapshots.

doc-drift

Diagnostic that computes deterministic doc-vs-code drift findings for one symbol. Reach for it to see exactly why `document/autodoc` considers a symbol's existing doc comment out of date.

Running it from the CLI

```
drag-lint doc-drift --qname X --db PATH [--json]
```

`--qname X` is the symbol to check. `--json` emits machine-readable output.

This is labeled a "diagnostic" in the CLI banner -- it is a troubleshooting tool, not a daily driver.

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

An index IS required. The project's DB lives at `<project folder>\_D-RAG\<project file>.sqlite`; run `drag-lint resolve-dbs --project <X.dproj>` if unsure.

Example

Illustrative only:

```
drag-lint doc-drift --qname CustomerOrder.TCustomerOrder.Total --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --json
```

This would report the specific drift findings between `Total`'s current doc comment and its code.

document-all

Documents every public declaration in every indexed unit, with no project scope. Reach for it to sweep DocInsight comments across the whole index at once, rather than one project at a time.

Running it from the CLI

```
drag-lint document-all [--stubs|--apply|--json|--no-backup|--include-accessors] [--db PATH]
```

Like the other batch document modes, the default is facts-only: an empty tag is never written, `<param>` never gets a skeleton, and `<summary>` appears only when harvested from a nearby `//` comment; add `--stubs` to keep a fresh comment that has no facts block. Trivial `Get*/Set*` property accessors are skipped by default; add `--include-accessors` to document them too. The usage line lists `--apply` as a flag; whether omitting it produces a dry run is not documented in the help text for this verb.

`document-all` covers every indexed unit with no project scope. This is different from `drag-lint document --project <p.dproj>`, which is scoped to the compile closure restricted to that project's own roots (declared in `_D-RAG\drag-lint-project.json`).

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

The `--db` flag is optional -- `drag-lint` auto-resolves the index when omitted. An index must still exist.

Example

Illustrative:

```
drag-lint document-all --apply --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would add facts-only DocInsight comments to every public declaration in every unit that this index covers, regardless of which project owns it.

dump-call-edges

Diagnostic that dumps every resolved call edge in the index as `ref_id|target_qname|confidence` rows. Reach for it when you need to inspect the raw call-resolution output behind `callgraph`, `find-callees`, or `call-path`.

Running it from the CLI

```
drag-lint dump-call-edges --db PATH
```

No further flags are documented in the usage line.

This is labeled a "diagnostic" in the CLI banner -- it is a troubleshooting tool, not a daily driver.

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

An index IS required. The project's DB lives at `<project folder>\_D-RAG\<project file>.sqlite`; run `drag-lint resolve-dbs --project <X.dproj>` if unsure.

Example

Illustrative only:

```
drag-lint dump-call-edges --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would print every resolved call edge in the index as `ref_id|target_qname|confidence` rows.

dump-refs

Diagnostic that dumps every ref in a file together with its `enclosing_symbol_id` attribution. Reach for it when a reference-derived answer (callers, doc facts, unused-symbol findings) looks wrong and you need to see the raw ref rows and which symbol each one was attributed to.

Running it from the CLI

```
drag-lint dump-refs <file> --db PATH
```

`<file>` is the source file to dump refs for. No further flags are documented in the usage line.

This is labeled a "diagnostic" in the CLI banner -- it is a troubleshooting tool, not a daily driver.

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

An index IS required. The project's DB lives at `<project folder>\_D-RAG\<project file>.sqlite`; run `drag-lint resolve-dbs --project <X.dproj>` if unsure.

Example

Illustrative only:

```
drag-lint dump-refs C:\Projects\MyApp\CustomerOrder.pas --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would print every ref found in `CustomerOrder.pas`, with the symbol ID each one was attributed to.

extract-method

Pulls a run of statements out of a routine into a new method. Reach for it when refactoring a long routine and you want the mechanical extraction done for you rather than by hand.

Running it from the CLI

```
drag-lint extract-method --file <F> --from-line <L1> --to-line <L2> --name <N> [--json|--apply|--no-backup]
```

--file is the source file; --from-line/--to-line bound the statement run to extract; --name is the new method's name. The usage line lists --apply as a flag; whether omitting it produces a dry run is not documented in the help text for this verb.

Reaching it in the IDE

The feature map's internal call site is `DragLint.Plugin.Keyboard.pas:319` -- a keyboard-bound plugin action, not a menu item. Beyond that call site, no menu path or keybinding is documented for this verb, so it is not stated here.

What it needs

No index required -- the usage line has no --db flag; extract-method operates directly on the given file's line range.

Example

Illustrative:

```
drag-lint extract-method --file C:\Projects\MyApp\Unit1.pas --from-line 120 --to-line 135 --name ValidateInput --apply
```

This would pull lines 120-135 of `Unit1.pas` into a new method named `ValidateInput`.

fb-snapshot

Connects to a Firebird database and captures a snapshot into a sqlite index. Reach for it when you need a drag-lint-queryable index built from a Firebird database connection.

Running it from the CLI

```
drag-lint fb-snapshot --connection "Database=...;User=...;Password=...;DriverID=FB" --db  
<sql.sqlite>
```

The `--connection` value's exact contents beyond the `Database=...;User=...;Password=...;DriverID=FB` shape shown in the usage line are not documented here -- supply your own real Firebird connection parameters in that form.

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

An index IS required -- the `--db` flag naming the target sqlite file is mandatory. Whether that file must already exist beforehand is not documented; run `drag-lint resolve-dbs` if unsure which DB is in play.

Example

Illustrative only:

```
drag-lint fb-snapshot --connection "Database=...;User=...;Password=...;DriverID=FB" --db  
sql.sqlite
```

This would snapshot the connected Firebird database into `sql.sqlite`.

find-callees

Lists the resolved outgoing calls of one routine. Reach for it to see what a routine actually calls, resolved through the call-edge index rather than by reading its source.

Running it from the CLI

```
drag-lint find-callees --qname <Foo.Bar> --db PATH [--json]
```

`--qname <Foo.Bar>` is the routine to inspect. `--json` emits machine-readable output.

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

An index IS required. The project's DB lives at `<project folder>\_D-RAG\<project file>.sqlite`; run `drag-lint resolve-dbs --project <X.dproj>` if unsure.

Example

Illustrative only:

```
drag-lint find-callees --qname CustomerOrder.TCustomerOrder.Total --db  
C:\Projects\MyApp\_D-RAG\MyApp.sqlite --json
```

This would list every routine that `Total` resolves calls to.

find-unit

Finds which indexed unit declares a symbol, and adds that unit to a file's `uses` clause. Reach for it when a symbol is undeclared and you want the fix looked up straight from the index.

Running it from the CLI

```
drag-lint find-unit --name <Symbol> --in <file> [--json|--apply|--no-backup] --db <db>
```

`--name` is the undeclared symbol; `--in` is the file to add the `uses` entry to. `--db` is required (not bracketed in the usage line). The usage line lists `--apply` as a flag; whether omitting it produces a dry run is not documented in the help text for this verb.

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature. It is NOT the same as the IDE's [Quick-Fix: Add Unit for Undeclared at Cursor](#) (Ctrl+Alt+U). The quick-fix parses a COMPILER DIAGNOSTIC, so it needs a prior build and a message that already names the unit. `find-unit --name <Symbol>` answers straight from the index instead, with no prior compile and no existing diagnostic message required -- it is strictly more capable here.

What it needs

An index IS required. The project's DB lives at `<project folder>\_D-RAG\<project file>.sqlite`; run `drag-lint resolve-dbs --project <X.dproj>` if unsure.

Example

Illustrative:

```
drag-lint find-unit --name TStringList --in C:\Projects\MyApp\Unit1.pas --apply --db  
C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would look up which indexed unit declares `TStringList` and add it to `Unit1.pas`'s `uses` clause.

ghost-check

Compiles a project with one or more units' content temporarily replaced by their unsaved editor buffers, then restores the original files unchanged. Reach for it to get compiler diagnostics against unsaved edits without saving them to disk.

Running it from the CLI

```
drag-lint ghost-check <dproj> ( --unit <real.pas> --buffer <buf> | --overlays <manifest> ) [--platform win32|win64] [--format json|text]
```

<dproj> is the project to compile. Give either a single --unit <real.pas> --buffer <buf> pair, or an -overlays <manifest> file listing multiple real-path/buffer-path pairs. --platform selects win32 or win64. --format selects json or text.

Reaching it in the IDE

No menu item calls this directly. The plugin's editor code (Editor.pas:2489) shells out to this verb to compile the active project against unsaved buffer content, backing "Compile Buffer (unsaved)" and feeding "Compile & Diagnose".

What it needs

Not needed, per the feature map's Index column.

Example

Illustrative only:

```
drag-lint ghost-check C:\Projects\MyApp\MyApp.dproj --unit  
C:\Projects\MyApp\frmOrders.pas --buffer C:\Projects\MyApp\_D-RAG\frmOrders.buf --  
platform win32 --format json
```

This would compile MyApp.dproj with frmOrders.pas's content replaced by the buffer file, report diagnostics as JSON, then restore frmOrders.pas unchanged.

ghost-recover

Scans a project's hidden `_D-RAG` folder for recovery journals left behind when a `ghost-check` overlay was interrupted by a crash, and restores each affected file to its saved original content. Reach for it (or let the IDE run it automatically) after a crash during a buffer-compile, so no unit is left holding overlaid (unsaved-buffer) content.

Running it from the CLI

Running the verb bare (no arguments) does not print a usage banner; it runs immediately and prints `ghost-recover: nothing pending. (exit 0)` when there is no recovery journal, or `ghost-recover: N file(s) restored. after acting.` Passing `--help` or any unrecognized flag produces `FATAL: Exception: Unknown argument: <flag> -- no other flags are documented.`

The plugin invokes it as:

```
drag-lint ghost-recover "<path-to-.dproj-or-folder>"
```

The single positional argument is optional; if omitted it defaults to the current directory.

Reaching it in the IDE

No menu item calls this by name directly; the feature map's "Recover Buffer-Compile Files" menu item is the user-facing action for it. The plugin's editor code (`Editor.pas:2522`) shells out to this verb, both on demand and automatically after a project loads, to restore any file left overlaid by a crashed `ghost-check`.

What it needs

Not needed, per the feature map's Index column.

Example

Illustrative only:

```
drag-lint ghost-recover C:\Projects\MyApp\MyApp.dproj
```

This would scan `C:\Projects\MyApp\_D-RAG` for `ghost-check` recovery journals and restore any affected file to its saved original content.

helpers-of

Lists record/class helper edges targeting a given type, anywhere in the index. Reach for it to see which helpers extend a type across the whole indexed codebase.

Running it from the CLI

```
drag-lint helpers-of <T> [--json] --db <db>
```

<T> is the target type name. --db is required (not bracketed in the usage line). This verb has no --apply flag -- it is a read-only listing.

Reaching it in the IDE

Reachable in the IDE only through the Structure form's right-click menu, not the main menu. The feature map's internal call site is `DragLint.Plugin.StructureForm.pas:1179`; the feature map does not name a specific menu-item label for it beyond that.

What it needs

An index IS required. The project's DB lives at `<project folder>\_D-RAG\<project file>.sqlite`; run `drag-lint resolve-dbs --project <X.dproj>` if unsure.

Example

Illustrative:

```
drag-lint helpers-of TColorKind --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would list every record/class helper in the index that targets `TColorKind`.

info

Prints engine self-info: version, build date, license (MIT), tree-sitter details, and capabilities. Reach for it to check which build of drag-lint you are running, or to script a version check.

Running it from the CLI

```
drag-lint info [--json]
```

Read-only.

Reaching it in the IDE

CLI+internal: the plugin calls this internally from `DragLint.Plugin.About.pas:217` (the About dialog). There is no menu item for it beyond that internal call site.

What it needs

No index required -- `info` reports on the engine binary itself, not on any indexed project.

Example

Illustrative:

```
drag-lint info --json
```

This would print the engine's version, build date, and capabilities as JSON.

link-orm

Links a project's code index to a SQL index so ORM-style relationships between Delphi symbols and SQL objects can be resolved across both. Reach for it when you want queries that cross from code into the schema it talks to.

Running it from the CLI

```
drag-lint link-orm --db <projDb.sqlite> --db <sqlDb.sqlite>
```

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

An index IS required -- in fact TWO: `link-orm` takes exactly two `--db` flags, one for the project's own DB and one for the SQL DB. The project's DB lives at `<project folder>\_D-RAG\<project file>.sqlite`; run `drag-lint resolve-dbs --project <X.dproj>` if unsure.

Example

Illustrative only:

```
drag-lint link-orm --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --db sql.sqlite
```

This would link `MyApp`'s code index to the `sql.sqlite` SQL index.

lint-project

Runs project-scoped structural lint rules against an index -- checks such as god-class, unused-public-symbol, circular-uses and layering-violation that need whole-project context rather than a single file. Reach for it for cross-file architectural findings, not per-file diagnostics.

Running it from the CLI

```
drag-lint lint-project --db <file.sqlite> [--rule god-class|unused-public-symbol|interface-reference-cycle|layering-violation|unused-private-member|unused-unit-in-uses|circular-uses|repeated-type-switch] [--layers <f.json>] [--json]
```

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

An index IS required. The project's DB lives at <project folder>_D-RAG\<project file>.sqlite; run `drag-lint resolve-dbs --project <X.dproj>` if unsure.

Example

Illustrative only:

```
drag-lint lint-project --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --rule circular-uses --json
```

This would report circular-uses findings for `MyApp` as JSON.

migrate-dbs

Moves project index databases into each project's own `_D-RAG` folder, matching the current index layout: a project's DB lives at `<project folder>\_D-RAG\<project file base name>.sqlite`. Only the per-platform library indexes stay in a shared folder. Reach for it after an index layout change, or to fix DBs stranded in an old shared location.

Running it from the CLI

```
drag-lint migrate-dbs [--config <drag-lint.json>] [--apply]
```

Without `--apply` this is a dry run -- it reports what it would move without moving anything.

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

No index required -- `migrate-dbs` operates on the index FILES themselves (moving them), not on their content, so there is nothing to query first.

Example

Illustrative:

```
drag-lint migrate-dbs --apply
```

This would move every project's index database into that project's own `_D-RAG` folder.

outline

Prints the file-scoped symbol outline for one Pascal unit. Reach for it when you want a unit's symbol list outside the IDE, or want to understand what feeds the Structure tree.

Running it from the CLI

```
drag-lint outline --file <path.pas> [--db <path>] [--format text|json]
```

--file <path.pas> is the unit to outline. --db <path> is optional; without it the exe falls back to its own default DB resolution. --format selects text or json.

Reaching it in the IDE

No menu item calls this directly. The plugin's structure cache (StructureCache.pas:338) shells out to this verb to populate the Structure form/tree used by "Show Structure" and the drag-lint Panel (dockable), caching the result per file.

What it needs

Optional, per the feature map's Index column.

Example

Illustrative only:

```
drag-lint outline --file C:\Projects\MyApp\frmOrders.pas --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --format json
```

This would print the symbol outline of `frmOrders.pas` as JSON -- the same data the Structure form displays as a tree.

pp-profile

Diagnostic that prints the resolved preprocessor define profile for a project, one symbol per line. Reach for it to see exactly which defines are active for a given project/platform/config combination, before trusting a `preprocess-file` or compile result.

Running it from the CLI

```
drag-lint pp-profile [--dproj PATH] [--platform win32|win64] [--config Release|Debug]
```

`--dproj PATH` selects the project. `--platform` selects `win32` or `win64`. `--config` selects `Release` or `Debug`.

This is labeled a "diagnostic" in the CLI banner -- it is a troubleshooting tool, not a daily driver.

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

No index required -- `pp-profile` takes no `--db` flag; it resolves the define profile directly from the project file.

Example

Illustrative only:

```
drag-lint pp-profile --dproj C:\Projects\MyApp\MyApp.dproj --platform win64 --config Debug
```

This would print every define active for `MyApp.dproj` under `Win64/Debug`, one per line.

preprocess-file

Diagnostic that prints a file's `{IFDEF}`-resolved source to stdout under a given define set. Reach for it to see exactly what the preprocessor produces for a file, without running a full compile.

Running it from the CLI

```
drag-lint preprocess-file --file PATH [--define SYM]... [--numeric K=V]... [--include-mode off|defines-only] [--no-near-search] [--tolerances]
```

`--file PATH` is the source file to preprocess. `--define SYM` may be given multiple times to set boolean defines. `--numeric K=V` may be given multiple times to set numeric defines. `--include-mode off|defines-only` controls `{I}` include handling. `--no-near-search` and `--tolerances` are present in the usage line but their behaviour is not documented beyond the flag name.

This is labeled a "diagnostic" in the CLI banner -- it is a troubleshooting tool, not a daily driver.

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

No index required -- `preprocess-file` takes no `--db` flag; it operates directly on the given file and defines.

Example

Illustrative only:

```
drag-lint preprocess-file --file C:\Projects\MyApp\CustomerOrder.pas --define DEBUG --numeric APIVERSION=2
```

This would print `CustomerOrder.pas` with `{IFDEF DEBUG}` blocks resolved and `APIVERSION` set to 2.

proptree

Recursive deep-property enumerator for a class: flattens its own and inherited properties into dotted paths, recursing into class-typed properties. Reach for it to see a class's full property surface, including nested sub-objects, without walking the class hierarchy by hand.

Running it from the CLI

```
drag-lint proptree --qname <X> [--depth N] [--no-to-persistent] [--refs-as-leaves] [--no-write-back] [--min-visibility published|public] [--format text|json] [--json] --db PATH [--db ...]
```

--qname <X> is the class to enumerate. --depth N bounds recursion. --no-to-persistent is present in the usage line; its effect is not documented beyond the flag name. --refs-as-leaves leaves TComponent-typed properties unexpanded (treated as references, not owned sub-objects). Types recovered by the ancestry-bridge are memoized back into the index automatically unless --no-write-back is given, which forces a read-only, non-mutating query. --min-visibility published|public filters emitted leaves by effective visibility (default: all). --format text|json selects output format; --db PATH may be repeated.

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

An index IS required. The project's DB lives at <project folder>_D-RAG\<project file>.sqlite; run drag-lint resolve-dbs --project <X.dproj> if unsure.

Example

Illustrative only:

```
drag-lint proptree --qname CustomerOrder.TCustomerOrder --depth 2 --min-visibility published --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --format text
```

This would print TCustomerOrder's published properties, up to 2 levels deep into any class-typed properties.

purge-locals

A size escape hatch for an index database that has grown too large: it drops local-variable and parameter symbols and runs VACUUM to reclaim space. Reach for it when a DB file is bigger than you want and you don't need per-local symbol lookups right now.

No data is lost for good: the call graph is unchanged, and the dropped local-variable/parameter rows are simply re-inflated the next time you index.

Running it from the CLI

```
drag-lint purge-locals --db PATH [--json]
```

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

An index IS required. The project's DB lives at `<project folder>\_D-RAG\<project file>.sqlite`; run `drag-lint resolve-dbs --project <X.dproj>` if unsure.

Example

Illustrative only:

```
drag-lint purge-locals --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --json
```

This would drop MyApp's local-variable/parameter symbol rows and VACUUM the database, without touching the call graph.

query ancestors

Resolves the transitive class/interface hierarchy of a type. Reach for it to see everything a type descends from -- or to check whether it descends from one specific ancestor -- read straight from the index.

Running it from the CLI

```
drag-lint query ancestors --name <type> [--of <ancestor>] [--db ...] [--json]
```

--name is the type to resolve. --of <ancestor> narrows the query to a specific ancestor.

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

The --db flag is optional -- drag-lint auto-resolves the index when omitted. An index must still exist.

Example

Illustrative:

```
drag-lint query ancestors --name TMyForm --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would list TMyForm's full transitive ancestor chain.

query find-callers

Lists callers of a routine by name, with an option to restrict results to precise, resolved callers. Reach for it to see who calls a given routine across the index.

Running it from the CLI

```
drag-lint query find-callers --name <callee-name> [--context N] [--resolved] [--db ...] [--json]
```

--name is the callee to look up. --context N adds N lines of surrounding context. --resolved restricts results to precise callers via resolved `call_edges`, grouped by target as certain or ambiguous.

Reaching it in the IDE

Not reachable through a main-menu item. The feature map's internal call site is `DragLint.Plugin.CodeLensCache.pas:466` -- the plugin's CodeLens cache, which powers inline caller-count annotations in the editor. Beyond that call site, no menu path or right-click item is documented for this verb.

What it needs

The --db flag is optional -- drag-lint auto-resolves the index when omitted. An index must still exist.

Example

Illustrative:

```
drag-lint query find-callers --name TMyClass.DoWork --resolved --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would list every resolved caller of `TMyClass.DoWork`, grouped as certain or ambiguous.

query typecat

Resolves a type's category -- float, string, class, interface, and so on. Reach for it when you need to know what kind of type a name refers to before writing code against it.

Running it from the CLI

```
drag-lint query typecat --name <type> [--db ...] [--json]
```

--name is the type to resolve.

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

The --db flag is optional -- drag-lint auto-resolves the index when omitted. An index must still exist.

Example

Illustrative:

```
drag-lint query typecat --name Currency --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would report which type category Currency resolves to.

safe-delete

Deletes a symbol if and only if it has zero references in the index. Reach for it to remove dead code with a guarantee against breaking a live caller.

Running it from the CLI

```
drag-lint safe-delete --name <QName> [--json|--apply|--no-backup] --db <db>
```

--name is the qualified symbol name to delete. --db is required (not bracketed in the usage line). The usage line lists --apply as a flag; whether omitting it produces a dry run is not documented in the help text for this verb.

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

An index IS required. The project's DB lives at <project folder>_D-RAG\<project file>.sqlite; run drag-lint resolve-dbs --project <X.dproj> if unsure.

Example

Illustrative:

```
drag-lint safe-delete --name Unit1.TMyClass.OldHelper --apply --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would delete TMyClass.OldHelper only if the index shows zero references to it.

schema

Prints the live schema of an index database: schema_version, every table and column, and row counts. Reach for it when you need to know exactly what an index DB contains before querying it, or to check its schema_version.

Running it from the CLI

```
drag-lint schema --db <file.sqlite> [--format text|json] [--output <file>]
```

Read-only.

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

An index IS required. The project's DB lives at <project folder>_D-RAG\<project file>.sqlite; run `drag-lint resolve-dbs --project <X.dproj>` if unsure.

Example

Illustrative only:

```
drag-lint schema --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --format json
```

This would print MyApp's index schema -- version, tables, columns, and row counts -- as JSON.

serve

Starts the MCP (Model Context Protocol) stdio server, for AI clients such as Claude or Cursor to query a drag-lint index.

serve is DIFFERENT from lsp: lsp is the LSP stdio server that the IDE plugin itself starts, and nothing in the IDE uses serve. If you are wiring drag-lint into an AI assistant, this is the verb; if you are wiring it into the RAD Studio IDE, it is lsp instead.

Running it from the CLI

```
drag-lint serve --db <file.sqlite>
```

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

An index IS required. The project's DB lives at <project folder>_D-RAG\<project file>.sqlite; run `drag-lint resolve-dbs --project <X.dproj>` if unsure.

Example

Illustrative only:

```
drag-lint serve --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This would start the MCP stdio server over MyApp's index, for an AI client to connect to.

shared-unit

Reads or extends the `dl:shared` marker on a unit, recording which projects share it. Reach for it when a unit is used by more than one project and you want that fact recorded in the source itself.

Running it from the CLI

```
drag-lint shared-unit --in <file.pas> [--add-project <name>] [--apply] [--json]
```

`--in` is the unit file. `--add-project <name>` adds a project name to the marker. Per the usage banner, this is a dry run without `--apply`: "read/extend the `dl:shared` marker; dry-run without `--apply`".

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

No index required -- the usage line has no `--db` flag; `shared-unit` reads and writes the `dl:shared` marker directly in the given file.

Example

Illustrative:

```
drag-lint shared-unit --in C:\Projects\Common\SharedUtils.pas --add-project MyApp --apply
```

This would add `MyApp` to `SharedUtils.pas`'s `dl:shared` marker.

usages

Finds usage sites of a named symbol, with a choice of report width. Reach for it when you want a usages listing outside the IDE, or want to understand what backs the "Usages" search kind in the Symbol Search dialog.

Running it from the CLI

```
drag-lint usages --name <X> [--width narrow|wide|very-wide] [--db <path>] [--depth N] [--format json]
```

--name <X> is the symbol to search for. --width controls report width (narrow, wide, or very-wide). --depth N is in the usage line with no further description -- not documented. --format json is the only format the usage line lists.

Reaching it in the IDE

No main-menu item calls this directly. The "Symbol Search..." dialog (drag-lint > Symbol Search...) has a "Kind" dropdown with Symbol, Text, and Usages options; choosing "Usages" and running a search shells out to this verb (source: SearchForm.pas:155).

What it needs

Optional, per the feature map's Index column.

Example

Illustrative only:

```
drag-lint usages --name TCustomerOrder --width wide --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite --format json
```

This would list usage sites of TCustomerOrder as JSON, at "wide" report width.

workspace add

Registers a project file into a `.drag-lint-workspace.json` workspace config, so multi-project workspace commands know about it. Reach for it when adding a new project to a workspace that `workspace index` / `workspace status` will operate over.

Running it from the CLI

```
drag-lint workspace add <projfile> [--config <.drag-lint-workspace.json>]
```

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

No index required -- `workspace add` only edits the workspace config file; it does not read or write an index database.

Example

Illustrative:

```
drag-lint workspace add MyProject.dproj
```

This would add `MyProject.dproj` to the workspace config in the current directory.

workspace index

Indexes every project registered in a `.drag-lint-workspace.json` workspace config in one pass. Reach for it to (re)build all of a workspace's project indexes together, rather than one project at a time.

Running it from the CLI

```
drag-lint workspace index [--config <.drag-lint-workspace.json>]
```

Reaching it in the IDE

CLI+internal: the plugin calls this internally from `DragLint.Plugin.ProjectNotifier.pas:103`. There is no menu item for it -- it is invoked in-process by the plugin, not exposed as a user-facing command.

What it needs

No index required to invoke the command -- `workspace index` is the command that BUILDS the workspace's project indexes; it reads the workspace config, not an existing index database.

Example

Illustrative:

```
drag-lint workspace index --config .drag-lint-workspace.json
```

This would index every project registered in that workspace config.

workspace status

Reports the status of every project registered in a `.drag-lint-workspace.json` workspace config. Reach for it to see, at a glance, which workspace projects are indexed, stale, or missing.

Running it from the CLI

```
drag-lint workspace status [--config <.drag-lint-workspace.json>]
```

Reaching it in the IDE

No IDE surface -- this is a CLI-only feature.

What it needs

No index required to invoke the command itself -- `workspace status` reads the workspace config and reports on each registered project's index state; it does not take a `--db` flag.

Example

Illustrative:

```
drag-lint workspace status
```

This would print the index status of every project registered in the workspace config found in the current directory.

Part IV -- Reference

rules

Lists the full lint rule catalog: every rule drag-lint can check, with its category, and its default enabled/fixable status. Reach for it to see what a lint run can find, or to filter by category.

The catalog currently holds 173 rules -- 149 enabled by default, 22 marked fixable -- across 16 categories. Of the 173, 119 are built-in and 54 are external `.scm` rule files.

Running it from the CLI

```
drag-lint rules [--json] [--category <name>] [--rules-dir <dir>]
```

Reaching it in the IDE

CLI+internal: the IDE's Options dialog loads this catalog by running `rules --json` internally, from `DragLint.Plugin.LintOptionsFrame.pas:88`. There is also a menu item that surfaces the catalog for one specific project: right-click a project in the Project Manager and choose "drag-lint: Project Rules..." -- see [drag-lint: Project Rules...](#).

What it needs

No index required -- `rules` reads the static rule catalog, not any project's indexed data.

Example

Illustrative:

```
drag-lint rules --category naming --json
```

This would print the catalog of naming-category rules as JSON.

lsp

Starts the LSP (Language Server Protocol) stdio server. This is what the IDE plugin itself launches to get hover, completion, go-to-definition and diagnostics inside the editor.

lsp is DIFFERENT from serve: serve is the MCP stdio server for AI clients such as Claude or Cursor, and nothing in the IDE uses serve. The IDE plugin uses lsp.

Running it from the CLI

```
drag-lint lsp --db <file.sqlite>
```

Reaching it in the IDE

CLI+internal: the plugin starts this process itself, from `DragLint.Plugin.LspClient.pas:297`. There is no menu item -- the IDE launches the lsp server automatically as a background stdio process, not via a user-facing command.

What it needs

An index IS required. The project's DB lives at `<project folder>\_D-RAG\<project file>.sqlite`; run `drag-lint resolve-dbs --project <X.dproj>` if unsure.

Example

Illustrative only:

```
drag-lint lsp --db C:\Projects\MyApp\_D-RAG\MyApp.sqlite
```

This is the same command the plugin runs in the background to serve the editor's language features for MyApp.

Feature Index

Every documented feature, by the surface it is reached from. Generated from the feature map, which is derived from the shipping build -- the CLI usage banner and the plugin's own menu registration.

See also [Features](#) for the grouped overview and [IDE Menu Reference](#) for the menu layout.

Main menu

- [Add Missing Units to uses \(whole unit\)...](#) -- check-unit
- [Auto-Document Whole Project...](#) -- index + document
- [Call Graph \(Butterfly\)...](#) -- reverse-calltree
- [Circular Uses Report \(cycles + fix plan\)...](#) -- cycles (worked example: [Circular Dependency Report](#))
- [Class Surface...](#) -- surface
- [Compile & Diagnose](#)
- [Compile Buffer \(unsaved\)](#)
- [Compiler Hints...](#) -- query hints
- [Copy Diagnostics \(Current File\)](#) -- lint
- [Doc Comment Stub \(symbol\)...](#) -- generate-docs
- [drag-lint Graph \(dockable\)](#)
- [drag-lint Options...](#)
- [drag-lint Panel \(dockable\)](#)
- [Export Enums \(Delphi const\)...](#) -- export enums
- [Export Graph \(DOT\)...](#) -- graph
- [Export to Obsidian...](#) -- export obsidian
- [Find Dead Code...](#) -- find-deadcode
- [Find Undocumented \(public\)...](#) -- query find
- [Find Usages...](#)
- [Format Whole Project with YADF...](#) -- format + index
- [Format with YADF](#) -- format
- [Full Compile Sweep](#) -- refresh-findings
- [Generate Test Helper CSV...](#) -- forms-csv
- [Go to Definition](#)
- [Hover at Cursor](#) -- hover
- [Impact / Blast Radius \(symbol\)...](#) -- impact
- [Import Build Log...](#) -- import-log
- [Library Drift Check...](#) -- library-drift
- [Lint Buffer \(Unsaved\)](#) -- lint
- [Open Plugin Log](#)
- [Quick-Fix: Add Unit for Inline Hint \(H2443\) at Cursor](#)
- [Quick-Fix: Add Unit for Undeclared at Cursor \(Ctrl+Alt+U\)](#)
- [Quick-Fix: Convert Public Field to Property at Cursor](#)
- [Rebuild Index for This Project](#) -- index
- [Reconcile Project Members \(.dpr/.dproj\)...](#) -- reconcile-project
- [Recover Buffer-Compile Files](#)
- [Rename Symbol...](#) -- rename

- [Reverse Call Tree \(clickable, Messages window\)...](#) -- reverse-calltree
- [Reverse Call Tree \(who calls this, N-deep\)...](#) -- reverse-calltree
- [Run AST Checks](#) -- check-ast
- [Run Diagnostics \(didSave\)](#)
- [Run Lint All \(Full Report\)...](#) -- lint-all
- [Scan TODOs / FIXMEs...](#) -- todos
- [Show Completion](#)
- [Show Resolved DBs \(debug\)...](#) -- resolve-dbs
- [Show Signature Help](#)
- [Show Structure](#)
- [Show Wiring \(Spring4D DI + DFM events\)...](#) -- wiring
- [Symbol Search...](#) -- query
- [Symbol Slice...](#) -- slice
- [Top Symbols \(fan-in\)...](#) -- top
- [Type at Cursor](#) -- typeat
- [Unit Test Stub \(symbol\)...](#) -- generate-test
- [Uses Audit -- interface->impl moves + unused \(this unit\)...](#) -- uses-audit
- [Uses Cleanup Preview \(compiler-verified, this unit\)...](#) -- uses-fix
- [Uses Report \(CSV\)...](#) -- uses-report

Right-click menus

- [Allow this message](#) -- allow
- [Copy All Diagnostics](#) -- lint
- [Create helper class](#) -- create-enum-helper
- [Document it](#) -- document
- [Document project](#) -- document
- [Document unit](#) -- document
- [drag-lint: Project Rules...](#) -- rules
- [Find Usages \(context\)](#) -- usages
- [Fix all in project](#) -- lint-all (autofix)
- [Fix all in unit](#) -- lint (autofix)
- [Fix it](#) -- lint (autofix)
- [Go to Declaration](#)
- [Go to Implementation](#)
- [Show in Call Graph](#) -- reverse-calltree

Tool windows

- [drag-lint](#)
- [drag-lint Graph](#)

CLI verbs

- [allow](#) -- allow
- [ambiguous-calls](#) -- ambiguous-calls
- [bench-context](#) -- bench-context

- [butterfly](#) -- butterfly
- [call-path](#) -- call-path
- [callgraph](#) -- callgraph
- [compile-check](#) -- compile-check
- [context](#) -- context
- [convert-apply](#) -- convert-apply
- [convert-scaffold](#) -- convert-scaffold
- [convert-validate](#) -- convert-validate
- [create-enum-helper](#) -- create-enum-helper
- [deps-report](#) -- deps-report
- [diff](#) -- diff
- [doc-drift](#) -- doc-drift
- [document-all](#) -- document-all
- [dump-call-edges](#) -- dump-call-edges
- [dump-refs](#) -- dump-refs
- [extract-method](#) -- extract-method
- [fb-snapshot](#) -- fb-snapshot
- [find-callees](#) -- find-callees
- [find-unit](#) -- find-unit
- [ghost-check](#) -- ghost-check
- [ghost-recover](#) -- ghost-recover
- [helpers-of](#) -- helpers-of
- [info](#) -- info
- [link-orm](#) -- link-orm
- [lint-project](#) -- lint-project
- [lsp](#) -- lsp
- [migrate-dbs](#) -- migrate-dbs
- [outline](#) -- outline
- [pp-profile](#) -- pp-profile
- [preprocess-file](#) -- preprocess-file
- [proptree](#) -- proptree
- [purge-locals](#) -- purge-locals
- [query ancestors](#) -- query ancestors
- [query find-callers](#) -- query find-callers
- [query typecat](#) -- query typecat
- [rules](#) -- rules
- [safe-delete](#) -- safe-delete
- [schema](#) -- schema
- [serve](#) -- serve
- [shared-unit](#) -- shared-unit
- [usages](#) -- usages
- [workspace add](#) -- workspace add
- [workspace index](#) -- workspace index
- [workspace status](#) -- workspace status

Circular Dependency Report

A worked example: a real four-unit uses cycle, and the report `drag-lint` produces for it.

****This page's report section is generated by drag-lint, not written by an AI.**** Every line under "The report, verbatim" below is the literal stdout of one command run against an indexed project. Every file name, line number and symbol name in it was read out of the index -- which was itself built by parsing the source with tree-sitter -- not inferred from prose. If you re-index the sample project and re-run the command, you will get byte-identical output.

The sample project

`circular-demo/` (repo root, alongside `README.md`) is a small, deliberately minimal Delphi console project: 4 units, ~75 symbols. Despite that size it contains a genuine four-unit dependency cycle that **still compiles**, because one of the four edges is an `implementation uses` rather than an `interface uses`:

```
DemoConfig  -> DemoLogger    interface
DemoLogger  -> DemoSession  interface
DemoSession -> DemoAudit    interface
DemoAudit   -> DemoConfig    IMPLEMENTATION <-- why this compiles at all
```

A cycle closed entirely through `interface uses` would not compile -- Delphi's compiler forbids it. This project is shaped so that exactly one edge is an `implementation uses`, which is legal, and that is the edge that lets the other three close the loop.

The project also declares seven module-level globals shared across the four units, each written by one unit and read by another:

Global	Declared in	Written by	Read by
<code>GDemoSettings</code>	<code>DemoConfig</code>	<code>DemoConfig</code>	<code>DemoAudit</code>
<code>GDemoMinLevel</code>	<code>DemoLogger</code>	<code>DemoConfig</code>	<code>DemoLogger</code> , <code>DemoConfig</code>
<code>GDemoLogCount</code>	<code>DemoLogger</code>	<code>DemoLogger</code>	<code>DemoConfig</code>
<code>GDemoSessionCount</code>	<code>DemoSession</code>	<code>DemoSession</code>	<code>DemoLogger</code>
<code>GDemoActiveSession</code>	<code>DemoSession</code>	<code>DemoSession</code>	<code>DemoLogger</code>
<code>GDemoAuditCount</code>	<code>DemoAudit</code>	<code>DemoAudit</code>	<code>DemoSession</code>
<code>GDemoAuditTrail</code>	<code>DemoAudit</code>	<code>DemoAudit</code>	<code>DemoSession</code>

Shared mutable globals are what make a cycle like this expensive in practice: they create ordering dependencies between units that the compiler will not complain about, so the cycle is not merely a tidiness problem -- it is a real coupling the reader has to hold in their head.

Reproduce it yourself

```
drag-lint index --project circular-demo\CircularDemo.dproj --db circular-demo\_D-
RAG\CircularDemo.sqlite
drag-lint cycles --db circular-demo\_D-RAG\CircularDemo.sqlite --edges --causes --plan --
format text
```

Indexing this project takes about 0.15 s (5 files, 73 symbols, 221 refs) -- small enough to run on a whim to check the claims on this page yourself.

The report, verbatim

```
# Cycle refactoring playbook
```

```
Generated by `drag-lint cycles --plan`. Recommendations are
best-effort -- the index can miss references (e.g. `set` types) -- so
**rebuild the project after each cycle** and re-run `drag-lint
cycles` to confirm the group is gone. If a build breaks, revert and
inspect by hand.
```

```
## Cycle 1: demologger <-> democonfig <-> demoaudit <-> demosession
```

```
Files:
```

```
- `demologger` -> `...\circular-demo\DemoLogger.pas`
- `democonfig` -> `...\circular-demo\DemoConfig.pas`
- `demoaudit` -> `...\circular-demo\DemoAudit.pas`
- `demosession` -> `...\circular-demo\DemoSession.pas`
```

```
### Why it cycles
```

```
- `demologger` interface uses `TDemoSession` (class) at `DemoLogger.pas:40`; declared in
`DemoSession.pas:20`.
- `democonfig` interface uses `TDemoLogLevel` (enum) at `DemoConfig.pas:23`; declared in
`DemoLogger.pas:15`.
- `demosession` interface uses `TDemoAuditKind` (enum) at `DemoSession.pas:35`; declared
in `DemoAudit.pas:24`.
```

```
### Recommended fix
```

```
**Extract the shared contract** into a new leaf unit both can depend on (it must use
NEITHER unit in this cycle).
```

```
Steps:
```

1. For each symbol under "Why it cycles", create/reuse a leaf unit (e.g. `- 2. Move the declaration there; add the new unit to the declaring unit's uses.
- 3. In each consumer, replace the cycle-partner in the **interface** uses with the new contracts unit.
- 4. Register the new unit in the .dpr/.dproj.

5. Rebuild the project (every project that compiles these units).
6. ****Verify:**** ``drag-lint cycles --db <db>`` -- cycle 1 should be gone.

The full writeup this page summarizes lives in the repository at [circular-demo/CYCLE-REPORT.md](https://github.com/circular-demo/CYCLE-REPORT.md).

What is worth noticing in that output

It names the exact symbol that forces each edge, not just the unit pair. "DemoLogger depends on DemoSession" is easy to say. Saying *why* -- `TDemoSession`, a class, referenced at `DemoLogger.pas:40`, declared at `DemoSession.pas:20` -- is what makes the cycle actionable. That is the difference between a warning and a fix plan, and it is only possible because the underlying index is AST-exact rather than a text/grep match.

It reports three edges, not four. The project has a four-unit cycle, but `drag-lint` lists only the three **interface** edges under "Why it cycles", because those are the ones that constrain compilation -- the ones you actually have to break. The fourth, implementation-level edge (`DemoAudit -> DemoConfig`) is exactly what makes the project legal Delphi in the first place, so it is reported separately (see `--edges`) rather than folded into the "why" list. The tool is not merely detecting a loop in a graph; it is distinguishing the edges that matter from the edge that makes the loop survivable.

The fix it proposes fits the shape of this cycle. All three offending symbols (`TDemoSession`, `TDemoLogLevel`, `TDemoAuditKind`) are type declarations, so "extract each into a leaf contracts unit" removes every interface edge without touching behavior. `--causes` and `--plan` are what produce this level of detail; see [Circular Uses Report \(cycles + fix plan\)](#) for the general feature page.

Honest caveats

- The report's own preamble says recommendations are best-effort and that the

index can miss references such as `set` types. It tells you to rebuild and re-run rather than trusting it blindly -- that caveat is `drag-lint`'s own, not added for this page.

- This project is deliberately small. It exists to make the output legible,

not to prove the tool scales -- for cycle reports on real, larger projects, run the same two commands against your own index.

See also

- [Features](#) -- the "Uses cycles" row in Graphs and reports links back here.
- [Circular Uses Report \(cycles + fix plan\)](#) -- the general `cycles` feature page (IDE menu item, all flags).
- [Uses Audit -- interface->impl moves + unused \(this unit\)](#) and [Uses Cleanup Preview \(compiler-verified, this unit\)](#) -- the compiler-verified follow-up once a cycle is diagnosed.